

**VISVESVARAYA TECHNOLOGICAL UNIVERSITY,
JNANASANGAMA, BELAGAVI – 590018**



A Internship Report

on

“REAL TIME CHAT APPLICATION”

Submitted in partial fulfillment for the award of degree of

Bachelor of Engineering

In

INFORMATION SCIENCE & ENGINEERING

Submitted by

DEEPAK DR 4MO21IS014

Internship Carried

Out at

THANIYA TECHNOLOGY

Internal Guide

Dr. JAYARAM CV

Professor

Department of CSE

MyCEM College, Mysore



External Guide

Mr. Vipul G Gajaragavonkar

Trainer

Thaniya Technology



**Department of Information Science & Engineering
MYSORE COLLEGE OF ENGINEERING & MANAGEMENT**

(Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi & Govt. of Karnataka)

#072, T Narsipura Road, Near Big Banyan Tree Chikkahalli, Mysore, Karnataka -28

DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING
MYSORE COLLEGE OF ENGINEERING AND MANAGEMENT

(Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi & Govt. of Karnataka)
#072, T Narsipura Road, Near Big Banyan Tree Chikkahalli, Mysore, Karnataka -28
2024-2025



CERTIFICATE

This is to certify that the Internship report entitled **“REAL TIME CHAT APPLICATION”** is a presentation done by **Mr. DEEPAK DR (4MO21IS014)** in partial fulfillment for the award of **Bachelor of Engineering** in **INFORMATION SCIENCE & ENGINEERING** of the Visvesvaraya Technological University, Belagavi, during the year 2024 - 2025. The Internship report has been approved as it satisfies the academic requirements in respect of seminar prescribed for the Bachelor of Engineering degree.

.....
Dr. JAYARAM CV

BE., M.Tech., Ph.D

Project Guide

Professor

Department of Computer Science & Engineering
MyCEM, Mysore – 570028

.....
Dr. JAYARAM C V

BE., M.Tech., Ph.D.

Project Coordinator

Professor

Dept. of CSE

MyCEM, Mysore – 570028

.....
Prof. HARSHAKUMAR H S

BE., M.Tech.

HOD

Dept. of CSE

MyCEM, Mysore – 570028

.....
Dr. M.S. PRABHUSWAMY

BE., M.Tech., Ph.D.

PRINCIPAL

MyCEM, Mysore – 570028

Name

Signature

1.Examiner-1		
2.Examiner-2		

UDYAM-KR-11-0010781

DATE: 15-05-2025

INTERNSHIP COMPLETION CERTIFICATE

Thaniya Technologies certifies that **Deepak DR (4MO21IS014)** from **Mysore College of Engineering and Management** had undertaken and completed the 16 weeks of Internship Program on **Full Stack Development** from **Feb 2025 to May 2025**.

Deepak DR displayed Professional traits during his internship period and managed to complete all assigned tasks as requested. He was hardworking, dedicated, and committed. It was a pleasure having him with us in this short period.

We wish him every success in life.

Best regards,

Sincerely,



Mr Dhanush

Project Manager



Kodikere, Kulai, Mangalore 575019



+91 7019582399



contact@thaniyatech.com
info@thaniyatech.com

DECLARATION

I Deepak DR, bearing USN: 4MO21IS014, studying in 8th semester in the Dept. of Information Science and Engineering at Mysore College Of Engineering and Management, Mysore, hereby declare that Internship Seminar entitled “Real Time Chat Application” which is being submitted by us in the partial fulfilment for the award of the degree of Bachelor of Engineering in Information Science & Engineering, from Visvesvaraya Technological University, Belagavi is an authentic record of us carried out during the academic year 2024-2025, under the guidance of Dr. Jayaram CV, Professor Department of CSE, Mysore College of Engineering and Management, Mysuru.

We further undertake that the matter embodied in the report has not been submitted previously for the award of any degree or diploma by us to any other university or institution.

Place: Mysuru

Date:

DEEPAK DR
(4MO21IS014)

ACKNOWLEDGEMENT

It is our pleasure to be indebted to various people, who directly or indirectly contributed in the development of this work and who influenced my thinking, behavior, and acts during the course of project.

We are extremely thankful to our principal **Dr. M S Prabhuswamy** for providing us the academic ambience and everlasting motivation to carry out this work and shaping our careers.

We express our sincere gratitude to **Mr. Harsha Kumar H S**, Head of Department, Computer Science & Engineering, MyCEM, for his stimulating guidance, continuous encouragement and motivation throughout the course of present work.

We wish to enunciate our special thanks to our project co-ordinator **Dr. Jayaram C V**, Professor, Department of Computer Science & Engineering, MyCEM, for his constant incitement and advice throughout the work.

We extend our sincere appreciation to our guide **Dr. Jayaram C V**, Professor, Department of Computer Science & Engineering, MyCEM, who provided his valuable suggestions and precious time in accomplishing our seminar.

We would also like to thank all other teaching and technical staffs of Department of Computer Science and Engineering, who have directly or indirectly helped us in the completion Of this seminar.

Lastly, we would like to thank the almighty and my parents for their moral support and my friends with whom we shared our day-to-day experience and received lots of suggestions that improved my quality of work.

DEEPAK DR (4MO21IS014)

ABSTRACT

The Real-Time Chat Application is a web-based communication platform developed to enable users to exchange messages instantly in both one-on-one and group settings. This application is built using the MERN stack—MongoDB, Express.js, React.js, and Node.js—with Socket.IO integration to facilitate real-time, bidirectional communication between clients and the server.

The core objective of this project is to deliver a seamless and responsive chatting experience that mirrors modern messaging applications. Key features include user authentication and authorization using JSON Web Tokens (JWT), chat room creation and management, real-time message transmission, and persistent message storage using MongoDB. The backend efficiently handles WebSocket connections and API requests, while the frontend offers a dynamic and interactive interface for users.

Security is enforced through encrypted password storage and token-based access control. Additionally, the application architecture ensures scalability, maintainability, and a clear separation of concerns between client and server components.

This project demonstrates the practical implementation of real-time web technologies and serves as a foundation for building scalable chat systems suitable for use in collaborative workspaces, customer support platforms, educational tools, and social media applications.

TABLE OF CONTENTS

	DECLARATION	i
	ACKNOWLEDGEMENT	ii
	ABSTRACT	iii
	LIST OF FIGURES	iv
CHAPTER 1	INTRODUCTION	1
	1.1 WEB DEVELOPMENT FUNDAMENTALS	1
	1.2 INTRODUCTION TO MERN STACK	1
	1.3 OBJECTIVES OF INTERNSHIP	2
	1.4 PERIOD OF INTERNSHIP	2
CHAPTER 2	COMPANY PROFILE	5
	2.1 INTRODUCTION	5-9
	2.2 ORGANIZATIONAL STRUCTURE	5
	2.2.1 PRINCIPLES	6
	2.3 VISION	6
	2.4 MISSION	6
	2.5 POLICIES	6-7
	2.6 PRODUCTS	8
	2.7 SERVICES	8-9
	2.8 CUSTOMERS	9
CHAPTER 3	INTERNSHIP PROGRESS	10
	3.1 FIRST WEEK	10
	3.1.1 INTRODUCTION TO HTML AND CSS	10
	3.1.2 HTML STRUCTURING THE WEB	10
	3.1.3 CSS (CASCADING STYLE SHEET)	11-13
	3.2 SECOND WEEK	13
	3.2.1 ADVANCED CSS CONCEPTS AND JAVASCRIPT	13
	3.2.1.1 BOX MODEL, TRANSFORMATION, AND DISPLAY	13-15
	3.2.2 JAVASCRIPT	15
	3.2.2.1 DATA TYPES IN JAVASCRIPT	15-17
	3.3 THIRD WEEK	18
	3.3.1 FUNCTIONS AND ARRAYS	18
	3.3.1.1 FUNCTIONS	18-20
	3.3.1.2 ARRAYS IN JAVASCRIPT	21-22
	3.4 FOURTH WEEK	22
	3.4.1 STRINGS IN JAVASCRIPT	22-24
	3.4.2 DESTRUCTURING	24
	3.4.2.1 ARRAY DESTRUCTURING	25
	3.4.2.2 OBJECT DESTRUCTURING	25
	3.4.3 JSON	25
	3.5 FIFTH WEEK	26
	3.5.1.1 INTRODUCTION TO DOM	26
	3.5.1.2 COMMON DOM METHODS	26-27
	3.5.1.3 TIME FUNCTIONS IN JAVASCRIPT	28-29

3.6 SIXTH WEEK	29
3.6.1 EXCEPTION HANDLING IN JAVASCRIPT	29
3.6.2 ASYNC AND AWAIT IN JAVASCRIPT	29-30
3.6.3 REACTJS	30
3.6.3.1 IMPORT AND EXPORT	30-31
3.6.3.2 PROPS	31
3.7 SEVENTH WEEK	32
3.7.1 KEY PROPS IN REACT	32
3.7.2 DEFAULT PROPS	32
3.7.3 CONDITIONAL RENDERING	32-33
3.7.4 REACT FRAGMENTS	33
3.7.5 HOOKS	33
3.7.5.1 USE STATE	34
3.8 EIGHTH WEEK	35
3.8.1 HOOKS CONTINUATION	35
3.8.1.1 USE REF	35-36
3.8.1.2 USE EFFECT	36-37
3.8.2 AXIOS METHOD	37
3.8.3 PROPS DRILLING	38-39
3.8.4 VIRTUAL DOM	39
3.8.5 FORM VALIDATION	40-41
3.9 NINTH WEEK	41
3.9.1 MONGODB	41-42
3.9.1.1 COMMANDS	42
3.9.2 CRUD OPERATIONS	42-44
3.9.3 COMPARISON OPERATORS	44-45
3.10 TENTH WEEK	45
3.10.1 QUERIES	45-46
3.10.2 SELECTION AND PROJECTION	46-47
3.10.3 ELEMENT OPERATORS	47
3.10.4 ARRAY OPERATORS	47
3.10.5 CURSOR OBJECT METHOD	47-48
3.10.6 UPDATE METHODS	48
3.11 ELEVENTH WEEK	49
3.11.1 UPDATE ARRAY OPERATORS	49
3.12 TWELFTH WEEK	49
3.12.1 NODE.JS	49-50
3.12.2 REPL	50-51
3.12.3 NODE JS ARCHITECTURE	51
3.12.4 MODULES	51-52
3.12.5 FILE SYSTEM	52-53
3.13 THIRTEENTH WEEK	54
3.13.1 HTTP	54-59
3.14 FOURTEENTH WEEK	59
3.14.1 EXPRESS JS	59
3.14.2 NODE PACKAGE MANAGER(NPM)	59-60
3.14.3 EXPRESSJS – ROUTING	60
3.14.4 EXPRESSJS – HTTP METHODS	60
3.14.5 EXPRESSJS – URL BUILDING	60-61

	3.15 FIFTEENTH WEEK	61
	3.15.1 WORKED ON PROJECT	61
CHAPTER 4	TASK COMPLETED	62
	4.1 CALCULATOR USING HTML AND CSS	62
	4.2 TIME TABLE USING HTML AND CSS	62-63
	4.3 REGISTRATION FORM USING HTML AND CSS	63
	4.4 BOUNCING BALL ANIMATION	64
	4.5 HEARTBEAT ANIMATION	64
	4.6 SQUARE BEAT ANIMATION	65
	4.7 HALCYONIC WEBSITE CLONE	66
	4.8 CHAMPIONS TROPHY	66-67
	4.9 DYNAMIC FOOD MENU DISPLAY WITH SEARCH FUNCTIONALITY	67-68
	4.10 DYNAMIC PRODUCT SHOWCASE WITH FETCH API	68-69
	4.11 SONG DISPLAY USING REACT	69
	4.12 LOGIN FORM USING REACT	70
	4.13 DESIGN A NAVIGATION BAR (NAVBAR) USING REACT	70-71
	4.14 TO-DO LIST USING REACT	71
CHAPTER 5	OUTCOMES	72
	5.1 MODULES AND IMPLEMENTATION	72-78
CHAPTER 6	CONCLUSION	79
	REFERENCE	80

LIST OF FIGURE

Fig.No	Title	Page.No
FIG 2.2	ORGANIZATIONAL STRUCTURE	5
FIG 3.8	VIRTUAL DOM	40
FIG 3.9	MONGODB DATABASE	42
FIG 3.12	NODEJS	50
FIG 4.1	CALCULATOR	62
FIG 4.2	TIME TABLE	63
FIG 4.3	REGISTRATION FORM	63
FIG 4.4	BOUNCING BALL	64
FIG 4.5	HEARTBEAT ANIMATION	65
FIG 4.6	SQUAREBEAT ANIMATION	65
FIG 4.7	WEBSITE CLONE	66
FIG 4.8	CHAMPION TROPHY PLAYCARD	67
FIG 4.9	DYNAMIC SEARCH BAR	68
FIG 4.9.1	RESULT AFTER SEARCH	68
FIG 4.10	DYNAMIC PRODUCTS SHOWCASE	69
FIG 4.11	SONG DISPLAY USING REACT	69
FIG 4.12	LOGIN FORM USING REACT	70
FIG 4.13	DESIGN A NAVIGATION BAR (NAVBAR) USING REACT	71
FIG 4.14	TO-DO LIST USING REACT USE STATE	71
FIG 5.1	USER LOGIN PAGE	73
FIG 5.2	USER REGISTRATION PAGE	74
FIG 5.3	USER CHAT APPLICATION DASHBOARD	74
FIG 5.4	USER PROFILE	75
FIG 5.5	CHAT FROM DEEPAK'S PERSPECTIVE	75
FIG 5.6	CHAT FROM SINCHANA'S PERSPECTIVE	76
FIG 5.7	GROUP CREATION INTERFACE IN CHAT APPLICATION	76
FIG 5.8	GROUP CHAT FROM SINCHANA'S PERSPECTIVE	77
FIG 5.9	DEEPAK'S VIEW - INITIATING THE CONVERSATION IN MERN GROUP CHAT	77
FIG 5.10	RAM OBSERVES – A THIRD-USER VIEW OF SYNCHRONIZED GROUP MESSAGING	78

CHAPTER 1

INTRODUCTION

1.1 WEB DEVELOPMENT FUNDAMENTALS

Web development is the process of building and maintaining websites and web applications. It encompasses various aspects including web design, front-end development, back-end development, and database management. The field has evolved significantly over the years, from simple static HTML pages to complex, interactive applications that provide rich user experiences.

HTML (Hypertext Markup Language) is the standard markup language used to structure content on the web. It provides the fundamental building blocks for web pages through a system of elements and tags that define different types of content such as headings, paragraphs, images, and links. HTML5, the latest version, introduced semantic elements like `<header>`, `<footer>`, `<article>`, and `<section>`, which provide more meaningful structure to web documents and improve accessibility and SEO.

CSS (Cascading Style Sheets) is a style sheet language used to describe the presentation of HTML documents. It controls the layout, colors, fonts, and overall visual appearance of web pages. CSS3, the current standard, introduced advanced features such as animations, transitions, gradients, and flexible box layouts (Flexbox) and grid layouts, which have revolutionized web design by enabling more sophisticated and responsive layouts without relying on JavaScript or other scripting languages.

JavaScript is a high-level, interpreted programming language that enables interactive web pages and is an essential part of web applications. Unlike HTML and CSS, which are primarily concerned with content structure and presentation, JavaScript adds behavior and functionality to web pages. It allows developers to create dynamic content, validate forms, handle events, manipulate the Document Object Model (DOM), and communicate with servers asynchronously. Modern JavaScript (ES6+) has introduced significant improvements such as arrow functions, template literals, destructuring, and promises, making code more concise and readable.

Together, HTML, CSS, and JavaScript form the foundation of front-end web development. They work in harmony to create the user-facing aspects of websites and web applications. HTML provides the structure, CSS enhances the visual presentation, and JavaScript adds interactivity and dynamic behavior. Mastering these three technologies is the first step in becoming proficient in the MERN stack, as they form the basis for more advanced frameworks and libraries like React.js.

1.2 INTRODUCTION TO MERN STACK

The MERN stack is a comprehensive JavaScript-based technology stack used for building modern web applications. It consists of four key components: MongoDB, Express.js, React.js, and Node.js. Each technology in the stack serves a specific purpose and together they enable developers to create full-stack applications using JavaScript throughout the entire development process.

MongoDB is a NoSQL database that stores data in flexible, JSON-like documents. Unlike traditional relational databases that use tables and strict schemas, MongoDB's approach allows for variable data structures and dynamic schema evolution. This flexibility makes it particularly suitable for applications where data requirements may change over time or where the data structure is not rigidly defined from the outset.

Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for building web and mobile applications. It simplifies the process of creating server-side applications by providing tools for routing, middleware integration, and HTTP request/response handling. Express.js follows the middleware pattern, allowing developers to create a pipeline of functions that process HTTP requests in sequence.

React.js is a JavaScript library developed by Facebook for building user interfaces. It uses a component-based architecture where UIs are composed of reusable, independent components. React's virtual DOM implementation efficiently updates the actual DOM by only rendering components that have changed, resulting in improved performance. This declarative approach to UI development makes code more predictable and easier to debug.

Node.js is a JavaScript runtime environment that executes JavaScript code outside a web browser. It uses an event-driven, non-blocking I/O model that makes it lightweight and efficient for data-intensive applications. Node.js enables server-side JavaScript execution, allowing developers to use the same programming language on both the client and server sides, thus streamlining the development process.

The MERN stack offers several advantages for web development. It allows for consistent use of JavaScript throughout the entire application, reducing the cognitive load of switching between different programming languages. The modular nature of the stack enables developers to work on individual components independently, facilitating parallel development and maintenance. Additionally, all components of the MERN stack are open-source and have large, active communities, ensuring ongoing support and a wealth of available resources and libraries.

1.3 OBJECTIVES OF INTERNSHIP

To master HTML5, CSS3, and modern JavaScript: The initial goal was to build a solid foundation in web development by thoroughly understanding HTML5 semantic elements, CSS3 layouts and animations, and ES6+ JavaScript features. This included learning about DOM manipulation, event handling, and asynchronous programming to create interactive web pages. The objective focused on writing clean, maintainable code that follows industry best practices.

To progress through the MERN stack technologies sequentially: This objective involves a structured learning path through each component of the MERN stack. After establishing front-end fundamentals, the plan is to learn Node.js and Express.js for backend development, followed by MongoDB for database operations, and finally React.js for building dynamic user interfaces. Each technology will be studied both independently and in the context of full-stack integration.

To develop practical projects that demonstrate increasing complexity: The internship aims to apply theoretical knowledge to practical projects of increasing sophistication. Beginning with static websites utilizing HTML and CSS, progressing to dynamic pages with JavaScript, then to server-side applications with Node.js, database-driven applications with MongoDB, and finally to full MERN stack projects with React frontends. Each project serves as a milestone to measure progress and competency.

To gain experience with modern development workflows and tools: This goal encompasses learning essential development tools and practices including version control with Git, package management with npm, debugging techniques, and testing methodologies. The objective includes becoming familiar with development environments, code editors, browser developer tools, and the command line interface to enhance productivity and code quality.

To build a professional portfolio of MERN stack projects: Throughout the internship, completed projects will be documented and compiled into a professional portfolio demonstrating MERN stack proficiency. This portfolio will showcase progressive skill development from basic HTML/CSS implementations to sophisticated full-stack applications, providing tangible evidence of capabilities to potential employers.

1.4 PERIOD OF INTERNSHIP

I completed a 15-week Full Stack Development Internship at Thaniya Technologies, Mangalore, Karnataka, from February 15, 2025, to May 15, 2025. This internship was an essential part of my academic curriculum and aimed to bridge the gap between academic learning and industry practices.

CHAPTER 2

COMPANY PROFILE

2.1 INTRODUCTION

Thaniya technologies is an IT solutions based company which specializes in performing custom design and development of projects which perfectly matches client business requirements. Their subsidiaries include Rooloo designs, Rooloo Fixme and Drony Aeriels. Their team uses cutting edge technologies to customize everything in a product development process whether it's about the look & feel feature of front-end to skilled back-end programming. Their customized methods and technologies result in full- functional, highly dynamic and interactive solution.

They offer variety of services such as web design, web development, app development, machine learning, Internet of things, game development and event management. With a team of qualified professionals that are dedicated to delivering quality services for the success of client's projects.

2.2 ORGANIZATIONAL STRUCTURE

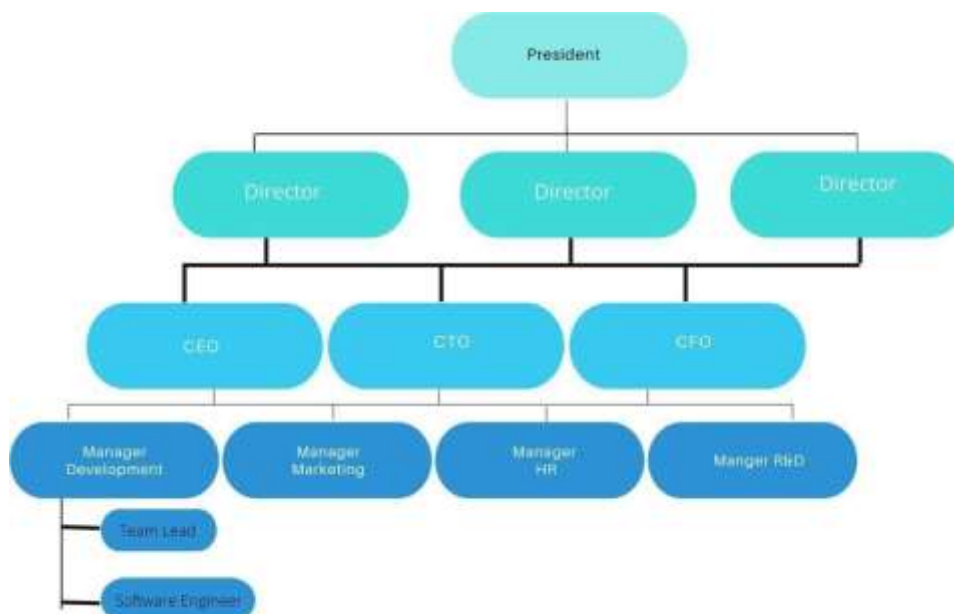


Fig 2.2: ORGANIZATIONAL STRUCTURE

2.2.1 PRINCIPLES

- **Interdependency:** Whole is greater than the sum of parts, knowing this has helped us to develop and cultivate a culture of interdependency. It portrays a power of connection and strives for the synergy through power of relationships. Interdependency between different departments, individuals, clients and partners for a greater good is what we are aiming at Thaniya technologies.
- **Sharing:** No social technology can grow without the benefit of sharing. The word sharing is thrown around with wild abandon by almost any organization you interact with, but very few works toward imbibing this concept in their products and services. At Techno Triumph, we value sharing and in fact have it a core concept in our internal as well as customer dealings. We also realize that sharing is a two-way street and the more you share something, the more you will receive from it.
- **Openness:** Openness as a business trait has gained a lot of momentum in recent times. Gone are the days when organizations were shy about revealing their business plans. Openness is intrinsic to all our business dealings. Advent of social media has brought about the change in the organizations and openness has become imperative. Openness becomes more important when dealing with digital ventures.

2.3 VISION

"To Provide Quality Service and Solution in the field of Information Technology"

2.4 MISSION

- Train future task force with Quality internship and training.
- Provide IT solution in various domain.
- Get Recognized by Government and other Quality Assurance bodies.

2.5 POLICIES

Talent Acquisition policies

Resume Screening and Short Listing

- The HR Department will do the initial screening & short listing of resumes for the position.

- HR Department during the short listing will focus on: Education, Experience, Current role/responsibilities, Salary level, Technical/ Functional/Domain Knowledge, Culture fit, etc.
- The short listed resumes are sent to the concerned Business/Unit/ Functional Heads for evaluation requesting them to indicate the action to be taken. As far as possible (depending on the availability), reasonable number of candidates will be provided for final selection.

Staffing Authority

The CEO/GCXO is the authority for staffing with the prior consent of Director.

Quality Policy General

As our first priority the management of Thaniya Technologies is committed to delivering quality software to our customers. We recognize that consistent satisfaction of customer needs is essential to business survival.

Planning

A Software Project Management Plan shall be created for all projects.

Monitoring and controlling

Project managers shall produce project performance reports at intervals agreed with the project sponsor.

Documenting requirements

All projects shall prepare a Software Requirements Specification describing the functions, performance and the interface requirements of the software product.

Architectural design

All projects shall conduct a Design Input Review to establish that design inputs such as Software

Requirements Specifications are unambiguous, complete and correct and possess sufficient quality to support the development of a design solution.

Coding

Projects shall develop software in compliance with predefined coding standards. Coding standards shall be updated to reflect any project specific practices.

Testing

All projects shall describe the approach to testing in a Software Test Plan. The STP shall describe the project's approach to unit, integration and acceptance testing.

Managing quality

All projects shall plan and perform the software quality management activities required to ensure that the customer's stated and implied needs are met and that the software product is developed in compliance with recognized best practice.

Managing people

Project managers shall be responsible for ensuring that development team members are aware of their quality responsibilities and appropriately trained to perform their assigned tasks.

2.6 PRODUCTS

Company develops android apps, web apps, ERP, Gaming etc. based on the customer requirements. So based on the needs of customer it develops the products.

2.7 SERVICES

App Development

Company develops Custom made Application for Business, if anyone have an idea join with company, it will build wonderful application for them.

Graphics Design

If anyone looking for Logo, Creative Poster, Visiting Cards, brochure etc. services regarding these can be provided.

Web Development

Are you Looking for Website, do you have a Business and you want to make it online, we will build website for you. Design-Host-Maintain.

Aerial Videography

Aerial Photography, Drone Photography/Videography can be provided to required ones.

Digital Marketing

It includes Search Engine Optimization, Translation and quality assurance of texts, Research work and keyword strategies.

Event Management

A-Z services for customer's event from Stage Decoration to Photography can be provided.

2.8 CUSTOMERS

Thaniya technologies served some companies/organizations such as Bhoomi Hardware, MCF, Payana, Redhill Softec, Namaste Online etc. by providing software solution in complete or part.

CHAPTER 3

INTERNSHIP PROGRESS

I learnt about the below-mentioned technologies and tools through my external guide and online resources. Apart from learning these new technologies and tools, I also learned how to effectively manage time and complete the given tasks within the deadline. The internship allowed me to develop my interpersonal skills.

3.1 FIRST WEEK:

3.1.1: INTRODUCTION TO HTML AND CSS

The first week of my internship laid the groundwork for understanding the basics of web development through an in-depth focus on HTML (HyperText Markup Language) and CSS (Cascading Style Sheets). These two technologies form the foundation for building and designing web pages, and mastering them is essential for creating effective and visually appealing websites.

3.1.2: HTML: STRUCTURING THE WEB

During this week, I gained hands-on experience with various key HTML elements:

- **Heading Tags:** I learned about the six levels of heading tags (<h1> to <h6>) and their role in defining the hierarchy and structure of a web page. I also practiced using appropriate headings to improve both readability and accessibility.
- **Image Tags ():** I explored the tag to embed images in web pages. This included understanding attributes like src (source) to specify the image path and alt to provide descriptive text, ensuring accessibility for users and search engines.
- **Anchor Tags (<a>):** I learned to create hyperlinks using the <a> tag. By working with the href attribute, I could link to other web pages, external resources, and even specific sections within a page.
- **Formatting Tags:** Practiced using tags such as , <i>, <u>, and to apply emphasis and improve content presentation.
- **Lists:** Worked with both ordered () and unordered lists () to organize information effectively. I also learned how to nest lists and add custom styles.

- **Tables:** Developed skills in creating tables using the `<table>` tag, along with attributes like `border`, `cellspacing`, `cellpadding`, and `align`, to organize data systematically.
- **Forms:** Learned to create interactive forms using the `<form>` tag. This included inputs for text fields, radio buttons, checkboxes, dropdown menus, and submit buttons.
- **Iframes:** Discovered how to embed external content like videos or web pages using `<iframe>` and its attributes for size and border control.
- **Semantic Tags:** Delved into semantic HTML tags such as `<header>`, `<footer>`, `<article>`, and `<section>` to create well-structured, meaningful, and accessible content for better SEO and user experience.

These elements helped me understand the fundamental building blocks of a web page and how they interact with each other to create meaningful content.

3.1.3: CSS (CASCADING STYLE SHEET)

In parallel, I studied both internal and external CSS to design and style web pages. Key learnings included:

- **Internal CSS:** Adding styles directly within the `<style>` tag in an HTML document to control the page's appearance.
- **External CSS:** Linking an external stylesheet using `<link>` for a more modular and reusable approach to styling.
- **Selectors and Properties:** Practiced applying various CSS selectors to target specific HTML elements and learned properties to modify layout, colors, font styles, and more.

CSS selectors are patterns used to target specific HTML elements for styling. I learned about the following types:

1. Simple Selectors

Simple selectors target specific elements based on their tag, class, or ID:

- **Tagname Selector:** Targets elements by their HTML tag (e.g., `p`, `h1`, `div`).
- **Class Selector:** Targets elements with a specific class attribute (e.g., `.className`).
- **ID Selector:** Targets elements with a specific id attribute (e.g., `#idName`).
- **Universal Selector:** Targets all elements (e.g., `*`).

2. PseudoClass

:link : Targets links that have not yet been visited.

- **Example:** `a:link { color: blue; }`

:visited : Styles links that the user has already visited.

- **Example:** `a:visited { color: purple; }`

:active : Styles an element while it is being clicked or activated.

- **Example:** `a:active { color: orange; }`

:focus : Targets an element that is focused (e.g., a form input or a link when navigated to using the keyboard).

- **Example:** `input:focus { border: 2px solid blue; }`

3. Pseudo Elements

::first-letter: Styles the first letter of a block-level element.

- **Example:** `p::first-letter { font-size: 2em; color: red; }`

::first-line: Styles the first line of a block-level element.

- **Example:** `p::first-line { font-weight: bold; }`

::before: Inserts content before the element's content.

- **Example:** `h1::before { content: "★ "; color: gold; }`

::after: Inserts content after the element's content.

- **Example:** `h1::after { content: " ★"; color: gold; }`

::selection: Styles the part of an element that the user highlights or selects.

1. **Example:** `::selection { background-color: yellow; color: black; }`

::marker: Targets the marker box of list items (like bullets or numbering).

- **Example:** `li::marker { color: blue; font-size: 1.5em; }`

4. Attribute Selectors

Attribute selectors target elements based on their attributes and values:

- **Presence:** [attribute] (e.g., [type] targets elements with a type attribute).
- **Equality:** [attribute="value"] (e.g., [type="text"] targets elements where type="text").

5. Combination Selectors

Combination selectors target elements based on relationships with other elements:

- **Descendant Selector:** Targets elements within a parent (e.g., div p).
- **Child Selector:** Targets direct children of a parent (e.g., div > p).

Responsive Design: Gained a basic understanding of using media queries to ensure web pages adapt seamlessly to different screen sizes and devices.

3.2 SECOND WEEK:

3.2.1 ADVANCED CSS CONCEPTS AND JAVASCRIPT

3.2.1.1 Box Model, Transformation, and Display

In the second week of my internship, I explored more advanced concepts in CSS, focusing on the box model, transformations, and the display property. These topics were crucial for understanding how to control and manipulate the layout and appearance of web elements effectively.

Box Model

The box model is a fundamental concept in CSS that defines the layout and spacing of elements on a web page. This week, I gained a thorough understanding of its components:

- **Content:** The inner part of the box where text and images are displayed.
- **Padding:** The space between the content and the border, used to provide breathing room inside the element.
- **Border:** The edge around the padding (if any) that outlines the element.
- **Margin:** The space outside the border, used to create distance between elements.

I practiced adjusting these properties using CSS to control the spacing and alignment of elements. Additionally, I learned about the box-sizing property, which allows developers to include or exclude padding and borders in the total width and height of elements.

Transformation

Transformations in CSS are used to rotate, scale, skew, or translate elements, creating dynamic and visually engaging effects. I explored the following transformation functions:

- **Translate:** Moves an element from its original position (transform: translateX(50px); or transform: translateY(20px);).
- **Scale:** Changes the size of an element (transform: scale(1.5); for enlarging or transform: scale(0.5); for shrinking).

Display Property

The display property is key to controlling the behavior of elements and their layout on a web page. I studied the following display values:

- **Block:** Elements take up the full width of their container, starting on a new line (display: block;).
- **Inline:** Elements appear on the same line as other content and take up only as much width as needed (display: inline;).
- **Inline-block:** Combines the features of inline and block elements, allowing elements to sit inline while respecting box model properties like padding and margins (display: inline-block;).

Text Properties

Includes properties like color, font-size, font-family, line-height, and text-align to control the appearance and alignment of text.

Marquee

The `<marquee>` tag creates scrolling text or images; however, it's outdated. Instead, CSS animations can achieve similar effects.

Positions

CSS provides positions like `static` (default), `relative` (offset from normal position), `absolute` (positioned relative to the nearest positioned ancestor), and `fixed` (stays in place relative to the viewport).

Transition Properties

Used to create smooth changes between property values, such as `transition: all 0.5s ease-in-out;` for animations like hover effects.

Animations

Created using `@keyframes` and `animation` properties, allowing elements to move, change colors, or transform dynamically (e.g., `animation: slide 2s infinite;`).

Background

CSS provides properties like `background-color`, `background-image`, and `background-size` to style an element's background.

Linear Gradient

A linear gradient creates a smooth transition between colors along a straight line. The direction can be specified (e.g., `top`, `left`, `diagonal`).

- Example: `background: linear-gradient(to right, red, blue);`

Radial Gradient

A radial gradient creates a transition between colors radiating outward from a central point, forming concentric circles.

- Example: `background: radial-gradient(circle, yellow, green);`

Navbar Styling

Styling a navigation bar involves flexbox or grid for alignment, background colors, hover effects, and proper padding/margins for clean and responsive designs.

3.2.2: JAVASCRIPT

It is both Frontend and Backend programming language which is used to develop web applications, mobile applications, server.

JavaScript is a object based programming language because everything is considered object in javascript.

3.2.2.1: Data Types in JavaScript:

JavaScript supports several data types:

1. Primitive Data Types:

- Number: Represents numeric values, e.g., 42, 3.14.
- String: Represents text, e.g., "Hello, World!".
- Boolean: Represents logical values, true or false.
- Undefined: A variable that has been declared but not assigned a value.
- Null: Represents an empty or non-existent value.
- Symbol: Introduced in ES6, it represents unique and immutable values.
- BigInt: Allows representation of integers larger than the Number type.

2. Non-Primitive (Object) Data Types:

- Objects: Collections of key-value pairs.
- Arrays: Ordered collections of values.
- Functions: JavaScript allows functions to be treated as objects.
- Date: Used for handling and manipulating date and time.

Keywords in JavaScript:

Keywords are reserved words that have specific meanings and uses in JavaScript:

- Examples: var, let, const, if, else, for, while, break, function, return, try, catch, switch, case, default, class, new, delete, typeof, etc.

Identifiers:

Identifiers are names used to identify variables, functions, or objects. They must begin with a letter, underscore (_), or dollar sign (\$), and cannot include reserved keywords.

Operators in JavaScript:

Operators are symbols used to perform operations on values and variables.

Types of Operators:

1. Arithmetic Operators: +, -, *, /, % (modulus), ** (exponentiation).
2. Comparison Operators: ==, !=, ===, !==, >, <, >=, <=.
3. Logical Operators: && (AND), || (OR), ! (NOT).
4. Assignment Operators: =, +=, -=, *=, /=.
5. Bitwise Operators: &, |, ^, ~, <<, >>.
6. Type Operators: typeof (returns the type of a variable), instanceof (checks if an object is an instance of a class).
7. Ternary Operator: condition ? expr1 : expr2.
8. Special Operators: delete (removes a property from an object), new (creates a new object).

Type of operator

The **typeof operator** in JavaScript is used to determine the data type of a given variable or value. It returns a string indicating the type of the operand.

- "undefined": If the variable is declared but has not been assigned a value.
- "string": For string values, e.g., "Hello".
- "number": For numeric values, e.g., 42 or 3.14.
- "boolean": For true or false.
- "object": For objects (including arrays and null).
- "function": For functions.
- "bigint": For large integer values, introduced in ES2020.
- "symbol": For unique and immutable values, introduced in ES6.

Example Usage:

```
console.log(typeof "hello"); // Output: "string"

console.log(typeof 123);    // Output: "number"

console.log(typeof true);   // Output: "boolean"

console.log(typeof {});     // Output: "object"

console.log(typeof null);   // Output: "object" (this is a known quirk in JavaScript)

console.log(typeof undefined); // Output: "undefined"

console.log(typeof function(){}); // Output: "function"
```

3.3 THIRD WEEK:

3.3.1: FUNCTIONS AND ARRAYS

3.3.1.1: Functions:

Functions are reusable blocks of code designed to perform a specific task. They make programs more modular, maintainable, and easier to debug. There are different types of functions in JavaScript.

1. Named Function

A named function has an explicit name that you use to call it. It's useful for reusability and clarity.

Example:

```
function greet(){

    console.log("Good Evening");

}

greet()
```

2. Function Expression

Functions can be assigned to a variable, creating a function expression. These functions are not hoisted.

Example:

```
let res=function add(){  
  
  console.log("hi i am new javascript")  
  
}  
  
res()
```

3. Anonymous Function

An anonymous function is a function without a name. It's often used as a function expression or in callbacks.

Example:

```
let res1 = function(){  
  
  console.log("Hi i am new hello world")  
  
}  
  
res1()
```

4. Arrow Function

Arrow functions provide a shorter syntax and were introduced in ES6. They do not bind their own this.

Example:

```
let add = ()=>2+2  
  
console.log(add());
```

5. Callback Function

A callback function is passed as an argument to another function and is executed later.

Example:

```
function add3(a,b){  
  console.log(a+b);  
}
```

```
add3(function test(){
    return 4
},4)

function add4(a,b){
    console.log(a()+b);
}

add4(function test(){
    return 4
},4
)
```

6. Higher-Order Function

A higher-order function takes another function as a parameter or returns a function.

Example:

```
function addition(a,b){
    console.log(a()+b);
}

function sub(){
    return 4
}

addition(sub,4)
```

7. Immediately Invoked Function Expression (IIFE)

An IIFE runs immediately after it is defined. It's wrapped in parentheses.

Example:

```
(function mm(){
    console.log("Hi i am javascript");
})();

(()=>{
    console.log("hi i am keerthana");
})()
```

3.3.1.2: Arrays in JavaScript:

An array is a special data structure used to store multiple values in a single variable. It allows you to work with an ordered collection of items.

Syntax:

```
let arrayName = [element1, element2, element3];
```

Array Methods:

Arrays in JavaScript come with a variety of methods for adding, removing, and manipulating elements:

1. Adding/Removing Elements:

- **push()**: Adds one or more elements to the end of an array.
- **pop()**: Removes the last element of an array.
- **unshift()**: Adds one or more elements to the beginning of an array.
- **shift()**: Removes the first element of an array.

2. Iterating over Arrays:

- **forEach()**: Executes a function for each element in the array.
- **map()**: Creates a new array with the results of calling a function on every array element.

3. Searching/Filtering:

- **indexOf()**: Finds the first occurrence of a value in the array.
- **includes()**: Checks if an array contains a certain element.
- **filter()**: Returns a new array with elements that pass a test function.

4. Sorting and Reversing:

- **sort()**: Sorts array elements alphabetically or using a compare function.
- **reverse()**: Reverses the order of the array.

5. Combining/Extracting Data:

- **concat()**: Combines two or more arrays into a new array.
- **slice()**: Extracts a portion of the array into a new array.

- **splice()**: Adds/removes elements from an array.

6. Transforming Arrays:

- **reduce()**: Reduces the array to a single value using a function.
- **join()**: Joins all array elements into a string.

3.4 FOURTH WEEK:

3.4.1: STRINGS IN JAVASCRIPT

A string is a sequence of characters enclosed within single quotes ‘’, double quotes “” or backticks ``.

Example:

```
let singleQuote = 'Hello';
```

```
let doubleQuote = "World";
```

Common String Methods

1. Length:

Returns the number of characters in a string.

Example:

```
let str = "Hello";
```

```
console.log(str.length); // Output: 5
```

2. Accessing Characters:

You can access characters using their index.

Example:

```
let str = "World";
```

```
console.log(str[0]); // Output: W
```

3. Case Transformation:

- **toUpperCase()**: Converts the string to uppercase.

- **toLowerCase():** Converts the string to lowercase.

Example:

```
let str = "Hello";  
console.log(str.toUpperCase()); // Output: HELLO
```

4. Trimming:

- **trim():** Removes whitespace from both ends of the string.

Example:

```
let str = " Hello ";  
console.log(str.trim()); // Output: "Hello"
```

5. Searching and Extracting:

- **indexOf():** Finds the index of the first occurrence of a substring.
- **includes():** Checks if a substring exists.
- **slice():** Extracts a section of the string.

Example:

```
let str = "JavaScript";  
console.log(str.indexOf("Script")); // Output: 4  
console.log(str.slice(0, 4)); // Output: Java
```

6. Substring and Substr:

- **substring(start, end):** Extracts a part of the string (excluding end).
- **substr(start, length):** Extracts a substring with a specified length.

Example:

```
let str = "JavaScript";  
console.log(str.substring(0, 4)); // Output: Java  
console.log(str.substr(4, 6)); // Output: Script
```

7. Concatenation:

Combines multiple strings.

Example:

```
let str1 = "Hello";  
  
let str2 = "World";  
  
console.log(str1 + " " + str2); // Output: Hello World
```

8. Splitting and Joining:

- **split()**: Splits a string into an array.
- **join()**: Joins an array into a string.

Example:

```
let str = "Hello World";  
  
let words = str.split(" ");  
  
console.log(words); // Output: ["Hello", "World"]
```

9. Replacing:

- **replace()**: Replaces a substring with another.

Example:

```
let str = "Hello, World!";  
  
console.log(str.replace("World", "JavaScript")); // Output: Hello, JavaScript!
```

10. Template Literals:

Introduced in ES6, template literals allow embedding expressions and variables.

Example:

```
let name = "Alice";  
  
let greeting = `Hello, ${name}!`;  
  
console.log(greeting); // Output: Hello, Alice!
```

3.4.2: DESTRUCTURING

Destructuring is a feature in JavaScript that simplifies extracting values from arrays or objects and assigning them to variables in a clear and concise way. It was introduced in ES6 (ECMAScript) and is widely used to work with complex data structures.

3.4.2.1: ARRAY DESTRUCTURING

Used to unpack values from arrays into variables.

Example:

```
let colors = ["red","green","yellow","white"]  
  
console.log(colors);  
  
let[a,b,c,d]=colors  
  
console.log(a,b,c,d);
```

3.4.2.2: OBJECT DESTRUCTURING

Used to unpack values from objects into variables.

Example:

```
let student = { name:"miller",age:35,dept:"CSE"}  
  
console.log(student);  
  
let[a,b,c,d]=student  
  
console.log(a,b,c,d);
```

3.4.3: JSON

JSON stands for JavaScript Object Notation. It is a lightweight data-interchange format that is easy for humans to read and write, and simple for machines to parse and generate. JSON is widely used to transmit data between a server and a web application or for storing structured data.

Features of JSON

1. Simple and Readable: It is text-based and easy to understand.
2. Language Independent: Although derived from JavaScript, JSON is supported by most programming languages, including Python, Java, Ruby, etc.
3. Flexible Data Representation: Handles objects, arrays, numbers, strings, booleans, and null values.

Json object

Example: {

```
  "name": "Alice",  
  "age": 25,  
  "isStudent": false  
}
```

3.5 FIFTH WEEK:

3.5.1: DOM

The Document Object Model (DOM) is a programming interface for web documents. It represents the structure of a document as a tree of nodes, allowing developers to manipulate the content, structure, and style of web pages dynamically using JavaScript.

3.5.1.1 : INTRODUCTION TO DOM

- The DOM treats an HTML or XML document as a hierarchical tree structure.
- Each element (e.g., <div>, <p>) is represented as a node within the DOM tree.
- With JavaScript, you can add, modify, or delete elements on the webpage using the DOM.

3.5.1.2 : COMMON DOM METHODS

DOM methods allow developers to interact with the structure and content of a web page.

1. Selecting Elements

- `getElementById()`: Selects an element by its ID.
- `document.getElementById("myId");`
- `getElementsByClassName()`: Selects elements by their class name.
- `document.getElementsByClassName("myClass");`
- `getElementsByTagName()`: Selects elements by their tag name.
- `document.getElementsByTagName("div");`
- `querySelector()`: Selects the first matching element using a CSS selector.
- `document.querySelector(".myClass");`
- `querySelectorAll()`: Selects all matching elements using a CSS selector.
- `document.querySelectorAll(".myClass");`

2. Modifying Elements

- `innerHTML`: Changes the content inside an element.
- `document.getElementById("myId").innerHTML = "New Content";`
- `textContent`: Changes the text content of an element.
- `document.getElementById("myId").textContent = "New Text";`
- `style`: Modifies the inline CSS of an element.

- `document.getElementById("myId").style.color = "red";`
- `setAttribute():` Sets an attribute (like `src`, `href`, `alt`) on an element.
- `document.getElementById("myImage").setAttribute("src", "image.jpg");`

3. Adding/Removing Elements

- `createElement():` Creates a new DOM element.
- `let newElement = document.createElement("p");`
- `newElement.textContent = "I am a new paragraph!";`
- `removeChild():` Removes a child element.
- `let parent = document.getElementById("parent");`
- `let child = document.getElementById("child");`
- `parent.removeChild(child);`

4. Event Handling

You can use DOM methods to add or remove event listeners on elements.

- `addEventListener():` Attaches an event handler.

Example:

```
document.getElementById("button").addEventListener("click", function() {  
    alert("Button clicked!");  
});
```

- `removeEventListener():` Removes an event handler.

Example:

```
document.getElementById("button").removeEventListener("click",  
myFunction);
```

- `Event Bubbling():` an event triggered on a child element first runs its handler, then propagates (or "bubbles") up the DOM tree to its parent elements and their handlers, in sequence.

Example:

```
<div class="grandparent" onclick="grandparent()">  
  <div class="parent" onclick="parent()">  
    <div class="child" onclick="child()"></div>  
  </div> </div>
```

```
function grandparent() {  
    console.log("Hi i am grandparent");}  
function parent(){  
    console.log("Hi i am parent");}  
function child(){  
    console.log("Hi i am child");  
}
```

3.5.1.3 : TIME FUNCTIONS IN JAVASCRIPT

JavaScript provides several time-related functions to execute code after a delay or repeatedly at regular intervals.

1. **setTimeout()**

Executes a function once after a specified delay (in milliseconds).

Example:

```
setTimeout(() => {  
    console.log("This runs after 2 seconds");  
}, 2000);
```

2. **clearTimeout()**

Stops a `setTimeout()` call that hasn't executed yet.

Example:

```
let timer = setTimeout(() => console.log("This won't run"), 2000);  
clearTimeout(timer);
```

3. **setInterval()**

Executes a function repeatedly at specified intervals.

Example:

```
let counter = 0;  
let interval = setInterval(() => {  
    counter++;  
    console.log(`Counter: ${counter}`);  
    if (counter === 5) clearInterval(interval);  
}, 1000); // Runs every second
```

4. clearInterval()

Stops a setInterval() call.

Example:

```
let interval = setInterval(() => console.log("Repeating..."), 1000);  
clearInterval(interval); // Stops the interval
```

3.6 SIXTH WEEK:

3.6.1: EXCEPTION HANDLING IN JAVASCRIPT

Exception handling is the process of detecting and managing errors during program execution. JavaScript provides powerful tools for gracefully handling unexpected issues to ensure the program continues to run smoothly.

Tools for Exception Handling:

Try Catch Block:

The try block contains code that might throw an error.

The catch block executes if an error is thrown.

Example:

```
try {  
  let result = 10 / 0;  
  console.log(result); // Output: Infinity  
} catch (error) {  
  console.log("Error occurred:", error.message);  
}
```

3.6.2: ASYNC AND AWAIT IN JAVASCRIPT

Async and Await are modern JavaScript features introduced for working with asynchronous operations, primarily promises. They simplify writing asynchronous code and make it easier to read and debug.

How Async/Await Works:

1. async Keyword:

Declares an asynchronous function that always returns a promise.

You can use await inside an async function.

Example:

```
async function fetchData() {  
  return "Data fetched successfully!";  
}  
  
fetchData().then(data => console.log(data));
```

2. await Keyword:

Pauses the execution of the async function until the promise is resolved or rejected.

Example:

```
async function getData() {  
  
  const response = await fetch("https://api.example.com/data");  
  
  const data = await response.json();  
  
  console.log(data);}   
  
getData(); // Fetches and logs the data when ready
```

3.6.3 REACTJS

ReactJS is a popular JavaScript library for building user interfaces, particularly for single-page applications. It allows developers to create reusable UI components and manage the state of applications efficiently.

3.6.3.1: IMPORT AND EXPORT

React uses ES6 modules for importing and exporting components, functions, or variables.

1. Export:

- Named Export:

```
export const name = "React";  
  
export const version = "18.0";
```

- Default Export:

```
const App = () => <h1>Hello, React!</h1>;  
  
export default App;
```

2. Import:

- Named Import:

```
import { name, version } from "./module";
```

- Default Import:

```
import App from "./App";
```

Named exports allow multiple exports from a file, while default exports are used for single exports.

3.6.3.2: PROPS

Props (short for properties) are used to pass data from a parent component to a child component in React. They are read-only and help make components reusable.

Syntax:

```
function Welcome(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}  
  
function App() {  
  return <Welcome name="React" />;  
}
```

Props can also be objects or variables, enabling dynamic data transfer between components.

3.7 SEVENTH WEEK:

3.7.1 KEY PROPS IN REACT

A key is a special prop that helps React identify which items in a list have changed, been added, or removed. Keys improve performance by allowing React to efficiently update the UI.

Example:

```
const numbers = [1, 2, 3, 4, 5];

const listItems = numbers.map((number) =>

<li key={number.toString()}>{number}</li>

);

return <ul>{listItems}</ul>;
```

3.7.2 DEFAULT PROPS

Default props allow you to set default values for a component's props in case they are not provided by the parent component. This ensures that your component behaves as expected even when some props are missing.

Example:

```
const UserInfo = ({ name = "N/A", age = "N/A" }) => (

<div>

<p>Name: {name}</p>

<p>Age: {age}</p>

</div>

);
```

3.7.3 CONDITIONAL RENDERING

It is the process of rendering the html element from the components based on the condition

Example:

```
function student() {

let marks = 20;

if (marks>50) {
```

```
    return <h1>Good Student</h1>;  
  }else{  
    return <h1>Bad Student</h1>;  
  } }
```

3.7.4 REACT FRAGMENTS

React Fragments allow you to group multiple elements without adding extra nodes to the DOM. Normally, when returning multiple elements from a component, you'd wrap them in a `<div>` but this can lead to unnecessary nesting in the DOM. Fragments solve this issue!

Example:

```
function MyComponent() {  
  return (  
    <React.Fragment>  
      <h1>Hello</h1>  
      <p>This is a paragraph.</p>  
    </React.Fragment>  
  );  
}
```

3.7.5 HOOKS

In React, a **hook** is a special function that allows functional components to use state and other React features without needing class components. Hooks were introduced in **React 16.8** to simplify component logic and improve reusability.

Rules of Hooks:

1. Hooks must be at the top level of functional component.
2. Hooks must be inside functional component.
3. Hooks can be inside custom hooks.
4. Hooks should not be inside class component.
5. Hooks should not be inside conditional statements.
6. Hooks should not be inside looping statements.
7. Hooks should not be inside regular functions.
8. Hooks should not be inside nested functions.

3.7.5.1: USE STATE():

The useState hook in React allows functional components to manage state. It provides a way to store and update values that persist across renders.

Key Features of useState

- Initializes state with a default value.
- Returns an array with the current state and a function to update it.
- Triggers re-renders when the state changes.

Example:

```
import React, {useState} from 'react'
import './Appss.css';
const Inputtracker = () => {
  const [inputValue, setInputValue] = useState("");
  const handleChange = (e) => {
    setInputValue(e.target.value);
  };
  return (
    <div className="container">
      <h1>Input Tracker</h1>
      <input
        type="text"
        className="input-field"
        placeholder="Type something..."
        value={inputValue}
        onChange={handleChange} />
      <p className="output">You typed: <span>{inputValue || '...'}</span></p>
    </div>
  )}
export default Inputtracker
```

3.8 EIGHTH WEEK:

3.8.1 HOOKS CONTINUATION

Hide and Show Button Example using useState:

```
import React from 'react';
import { useState } from 'react';
const Hooks = () => {
  let [toggle, setToggle] = useState(true);
  let hideHeading = () => {
    setToggle(false);
  }
  let showHeading = () => {
    setToggle(true);
  }
  return (
    <div>
      {toggle === true && <h1>Hi welcome to internship</h1>}
      <button onClick={showHeading}>Show</button>
      <button onClick={hideHeading}>Hide</button>
    </div>
  );
}
export default Hooks;
```

3.8.1.1: USE REF():

- useRef is a Hook that allows to modify or create a reference to the DOM element in the functional Component.
- useRef is used for focusing and media purpose.
- The main use of useRef Hook is to access the DOM elements in a more efficient way.
- useRef will return one object with current value.

Example:

```
import React, { useRef } from "react";

function NameInput() {
  const inputRef = useRef(null);

  const focusInput = () => {
    if (inputRef.current) {
      inputRef.current.focus(); // Focus the input field
    }
  };

  return (
    <div>
      <input ref={inputRef} type="text" placeholder="Enter your name" />
      <button onClick={focusInput}>Focus Input</button>
    </div>
  );
}

export default NameInput;
```

3.8.1.2: USE EFFECT():

- The useEffect in the ReactJs is used to handle the sideeffect such as fetching the data and updating DOM.
- This hooks runs on every render but there is also way of using dependency array using which we can control the effect of re-rendering.

Example:

```
import React, { useRef, useEffect } from "react";

function NameInput() {
  const inputRef = useRef(null);

  useEffect(() => {
    if (inputRef.current) {
      inputRef.current.focus();
    }
  });
}
```

```
    }  
    }, []); // Empty dependency array ensures it runs only once after mount  
    return (  
      <div>  
        <input ref={inputRef} type="text" placeholder="Enter your name" />  
        <button onClick={() => inputRef.current.focus()}>Focus Input</button>  
      </div>  
    );  
  }  
  export default NameInput;
```

3.8.2 AXIOS METHOD

Axios is a **JavaScript library** used to make HTTP requests from the browser or Node.js. It helps in **fetching data** from an API, **sending data** to a server, and **handling responses** efficiently.

Example:

```
import React, { useState, useEffect } from "react";  
import axios from "axios";  
function FetchData() {  
  const [data, setData] = useState(null);  
  useEffect(() => {  
    axios.get("https://jsonplaceholder.typicode.com/users")  
      .then(({ data }) => setData(data))  
      .catch(console.error);  
  }, []);  
  return data ? <ul>{data.map((user) => <li key={user.id}>{user.name}</li>)}</ul> :  
    <p>Loading...</p>;  
}  
export default FetchData;
```

3.8.3 PROPS DRILLING

Props Drilling happens in React when you pass down props through multiple nested components just to get data to a deeply nested child component. This can make your code hard to maintain.

Example:

```
import React from "react";

// Parent component

function App() {

  const user = { name: "Alice", age: 25 };

  return <Parent user={user} />;

}

// Intermediate component passing the prop further

function Parent({ user }) {

  return <Child user={user} />;

}

// Deeply nested component

function Child({ user }) {

  return <p>Name: {user.name}, Age: {user.age}</p>;

}

export default App;
```

If you have **many nested components**, passing props manually becomes repetitive.

Solution: Context API

Instead of passing props down manually, **React Context API** allows components to access global data **without drilling**

Example:

```
import React, { createContext, useContext } from "react";

const UserContext = createContext();

function App() {

  const user = { name: "Alice", age: 25 };


```

```
    return (  
      <UserContext.Provider value={user}>  
        <Child />  
      </UserContext.Provider>  
    );  
  }  
  
  function Child() {  
    const user = useContext(UserContext);  
    return <p>Name: {user.name}, Age: {user.age}</p>;  
  }  
  
  export default App;
```

3.8.4 VIRTUAL DOM

The **Virtual DOM** is a key concept in React that helps improve performance and efficiency when updating the UI. Instead of directly modifying the real DOM, React creates a **lightweight copy** of it, known as the **Virtual DOM**, where changes are first applied before updating the actual DOM.

How the Virtual DOM Works:

1. **Initial Render:** React creates a Virtual DOM representation of the UI.
2. **State/Prop Change:** When data changes, React updates the Virtual DOM instead of modifying the real DOM immediately.
3. **Diffing Algorithm:** React compares the previous Virtual DOM with the updated one to detect differences.
4. **Efficient Updates:** React only updates the necessary parts of the actual DOM, reducing performance overhead.

Benefits of the Virtual DOM:

- **Faster updates:** Avoids unnecessary manipulations of the real DOM.
- **Efficient rendering:** Only changes are applied, rather than reloading the entire UI.
- **Improved performance:** Reduces costly operations and speeds up UI rendering.

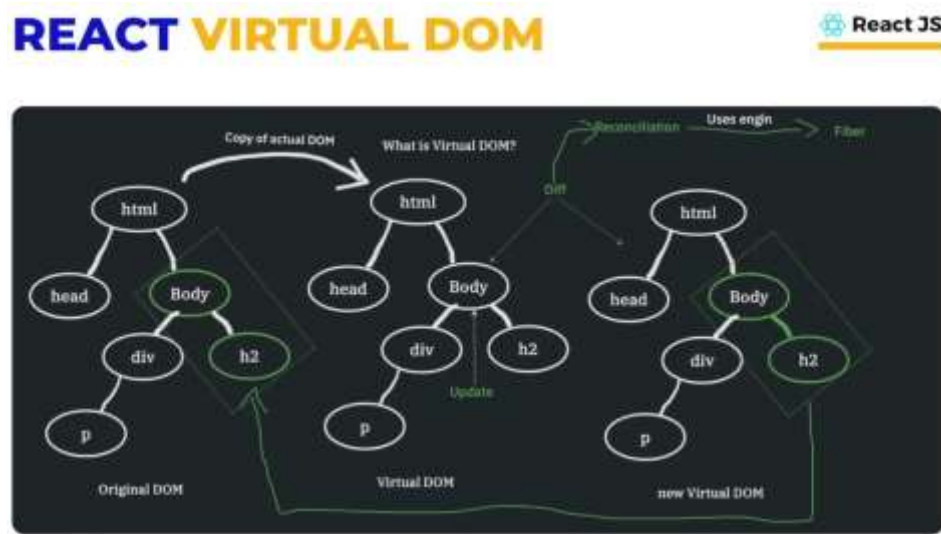


Fig 3.8: Virtual DOM

3.8.5 FORM VALIDATION

Form validation is the process of checking user input in a form before submitting it. It ensures that the provided data is **correct, complete, and follows expected rules** to prevent errors or security issues.

Example:

```
import React, { useState } from "react";

function FormValidation() {
  const [email, setEmail] = useState("");
  const [error, setError] = useState("");
  const validateEmail = (e) => {
    const value = e.target.value;
    setEmail(value);
    // Basic email validation
    if (!value.includes("@") || !value.includes(".")) {
      setError("Invalid email format");
    } else {
      setError("");
    }
  }
}
```

```
};

const handleSubmit = (e) => {
  e.preventDefault();
  if (!email) {
    setError("Email is required");
  } else if (!error) {
    alert("Form submitted successfully!");
  }
};

return (
  <form onSubmit={handleSubmit}>
    <input type="text" placeholder="Enter email" value={email}
onChange={validateEmail} />
    {error && <p style={{ color: "red" }}>{error}</p>}
    <button type="submit">Submit</button>
  </form>
);
}

export default FormValidation;
```

3.9 NINTH WEEK

3.9.1 MONGODB

MongoDB is a popular NoSQL database that stores data in a flexible, JSON-like format called BSON (Binary JSON). Unlike traditional relational databases (such as MySQL or PostgreSQL), MongoDB does not use tables and rows. Instead, it works with documents and collections, making it highly scalable and efficient for handling large amounts of unstructured data.



Fig 3.9: MongoDB Database 27017

3.9.1.1: COMMANDS

- **showdbs:** show dbs lists all databases in the MongoDB server.
- **use:** This command switches to the specified database.
 - If the database does not exist, MongoDB creates it automatically when you add data.

Example:

use myDatabase

- This switches to (or creates) a database called myDatabase.

1. Command to create collection:

```
db.createCollection("name")
```

2. Command to display the list of collection:

```
show collections
```

3. Command to delete database:

```
db.dropDatabase( )
```

4. Command to delete collection:

```
db.collectionname.drop( )
```

3.9.2 CRUD OPERATIONS

CRUD operations (Create, Read, Update, Delete) are the basic actions you can perform on a MongoDB database.

1. Create Method:

- **insertOne(data):** Inserts a single document into a collection.

Example:

```
db.users.insertOne({ name: "Alice", age: 25 })
```

- **insertMany(data):** Inserts multiple documents at once.

Example:

```
db.users.insertMany([ { name: "Bob", age: 30 },  
  { name: "Charlie", age: 28 } ])
```

2. Read Method:

- **findOne(filter,options):** Returns only one document that matches the query.

Example:

```
db.users.findOne({ name: "Alice" })
```

If multiple documents match, it **returns the first one** found.

- **findMany(filter,options):** Returns multiple documents that match the query. Can be used with filters and projections.

Example:

```
db.users.find({ age: { $gt: 25 } }) # Get users older than 25
```

3. Update Method:

- **updateOne(filter,data,options):** Updates only one document that matches the filter. If multiple documents match, **only the first one** gets updated.

Example:

```
db.users.updateOne({ name: "Alice" }, { $set: { age: 26 } })
```

- **updateMany(filter,data,options):** Updates multiple documents that match the filter.

Example:

```
db.users.updateMany({ city: "New York" }, { $set: { country: "USA" }  
  })
```

- **replaceOne(filter,data,options):** Replaces an entire document, keeping only the new fields. Unlike updateOne(), existing fields are removed unless included.

Example:

```
db.users.replaceOne(  
  { name: "Bob" },  
  { name: "Bob", age: 30, city: "Chicago" }  
)
```

4. Delete Method:

- **deleteOne(filter):** Deletes only one document that matches the filter

Example:

```
db.users.deleteOne({ name: "Alice" })
```

- **deleteMany(filter):** Deletes multiple documents matching the filter.

Example:

```
db.users.deleteMany({ city: "New York" })
```

3.9.3 COMPARISON OPERATORS

1. **\$eq** → Equal to
2. **\$ne** → Not equal to
3. **\$gt** → Greater than
4. **\$gte** → Greater than or equal to
5. **\$lt** → Less than
6. **\$lte** → Less than or equal to
7. **\$in** → is used to compare multiple value of same field
8. **\$nin** → not in

Examples for the comparison operators:

1. Write a query to display all the documents who is getting some salary.
⇒ `db.employee.find({"salary": {$ne: null}})`
2. Write a query to display all the employee details who is working in dept HR or Finance.
⇒ `db.employee.find({"dept": {$in: ["HR", "Finance"]}})`
3. Write a query to display all the employee details whose salary is more than 10,000.
⇒ `db.employee.find({"salary": {$gt: 10000}})`
4. Write a query to display where dept is IT.
⇒ `db.employee.find({"dept": "IT"})`
5. Details whose is not working under dept "HR" & IT.
⇒ `db.employee.find({"dept": {$nin: ["HR", "IT"]}})`
6. Write a query to display all the employee details whose is not getting salary less than 15000.
⇒ `db.employee.find({"salary": {$not: {$lt: 15000}}})`
7. Write a query to display all the employee details who is getting salary below 15000.

⇒ db.employee.find({"salary": {\$lt: 15000}})

8. Write a query to display all the employee details who is working in department IT and having the salary less than 25000

⇒ Db.employee.find({"dept":{"\$eq: "IT"}, "salary":{"\$lt:25000}})

3.10 TENTH WEEK

3.10.1 QUERIES

1. Write a query to display all the employee details who is working in dept HR OR the emp having age 50.

⇒ db.employee.find({\$or: [{"dept": "HR"}, {"age": 50}]})

2. Write a query to display all the employee details who is working under IT and getting salary 20000 or working in dept production and getting salary 22000.

⇒ db.employee.find({\$or: [{"dept": "IT", "salary": 20000}, {"dept": "production", "salary": 22000}]})

3. Write a query to display all the employee details who is working in dept sales or employee department gender male.

⇒ db.employee.find({\$or: [{"dept": "sales"}, {"gender": "male"}]})

4. Write a query to display all the employee details except dept HR and IT

⇒ db.employee.find({\$nor: [{"dept": "HR"}, {"dept": "IT"}]})

5. Write a query to display all the employee details who is working as production and his gender is male.

⇒ db.employee.find({\$and: [{"dept": "Production"}, {"gender": "male"}]})

6. Write a query to display all the employee details who is getting salary below 15000 or dept is production.

⇒ db.employee.find({\$or: [{"salary": {\$lt: 15000}}, {"dept": "production"}]})

7. Write a query to display all the employee details who is not getting any salary or who is not working in any dept.

⇒ db.employee.find({\$or: [{"salary": null}, {"dept": null}]})

8. Write a query to display all the employee details whose age is greater than 30 and getting some salary.

⇒ db.employee.find({\$and: [{"salary": {\$ne: null}}, {"age": {\$gt: 30}]})

9. Write a query who is getting salary more than 30,000 and working in dept sales or dept IT.

⇒ `db.employee.find({$and: [{"salary": {$gt: 30000}}, {$or: [{"dept": "sales"}, {"dept": "IT"}]}])`

10. Write a query who is not getting salary and who is not working in any department.

⇒ `db.employee.find({$and: [{"salary": null}, {"dept": null}])`

3.10.2 SELECTION AND PROJECTION

Selection:

Selection refers to **filtering documents** from a collection based on specific conditions. It is done using the `find()` method.

Example:

`db.users.find({ age: 25 })` # Selects users with age 25

Projection:

Projection controls which fields should be displayed in the query result. It is defined in the second parameter of `find()`.

Example:

`db.users.find({}, { name: 1, age: 1 })` # Shows only name & age

`db.users.find({ city: "New York" }, { _id: 0, name: 1 })` # Hides `\_id`

Example Queries:

1. Show only emp names and dept.
⇒ `db.employee.find({}, {name: 1, dept: 1, _id: 0})`
2. Show names and salary of all emp.
⇒ `db.employee.find({}, {name: 1, _id: 0, salary: 1})`
3. Show name, age and gender of all emp. `db.employee`.
⇒ `find({}, {name: 1, _id: 0, age: 1, gender: 1})`
4. Show department and salary of only male employees.
⇒ `db.employee.find({gender: "male"}, {dept: 1, _id: 0, salary: 1})`
5. Show only names of emp from the HR dept. `db.employee`.

⇒ `find({dept: "HR"}, {name: 1, _id: 0})`

6. Show names and salary of employees who is having some salary.

⇒ `db.employee.find({salary: {$ne: null}}, {name: 1, _id: 0, salary: 1})`

7. Show the emp names and salary who is getting salary more than 15000.

⇒ `db.employee.find({salary: {$gt: 15000}}, {name: 1, _id: 0, salary: 1})`

3.10.3 ELEMENT OPERATORS

1. **\$exists:** Checks if a field exists in a document.

Example:

`db.users.find({ age: { $exists: true } })` # Selects users with an "age" field

2. **\$type:** Matches documents where a field is a specific data type.

Example:

`db.users.find({ age: { $type: "number" } })` # Finds users where "age" is a number

3.10.4 ARRAY OPERATORS

1. **\$size:** Matches arrays with a specific number of elements.

Example:

`db.users.find({ skills: { $size: 3 } })` # Finds users with exactly 3 skills

2. **\$all:** Matches arrays containing all specified values.

Example:

`db.users.find({ hobbies: { $all: ["reading", "traveling"] } })` # Must have both hobbies

3.10.5 CURSOR OBJECT METHOD

1. **Count():** Returns the total number of matching documents.

Example:

`db.users.find({ age: 25 }).count()`

2. **Skip(n):** Skips the first n documents in the query result.

Example:

`db.users.find().skip(2)`

3. **Sort():** Orders documents in ascending (1) or descending (-1) order.

Example:

`db.users.find().sort({ age: -1 })` # Sort by age in descending order

4. **Limit(n):** Restricts the number of documents returned.

Example:

```
db.users.find().limit(5)
```

3.10.6 UPDATE METHODS

In MongoDB, the update method is used to modify documents, and various update operators help control specific field changes.

1. **\$set – Updates or adds a field**

Example:

```
db.users.updateOne({ name: "Alice" }, { $set: { age: 26 } })
```

2. **\$unset – Removes a field**

Example:

```
db.users.updateOne({ name: "Alice" }, { $unset: { age: "" } })
```

3. **\$min – Updates the field only if the new value is lower**

Example:

```
db.users.updateOne({ name: "Alice" }, { $min: { age: 24 } })
```

4. **\$max – Updates the field only if the new value is higher**

Example:

```
db.users.updateOne({ name: "Alice" }, { $max: { age: 30 } })
```

5. **\$mul – Multiplies the field value by a factor**

Example:

```
db.products.updateOne({ item: "Laptop" }, { $mul: { price: 1.10 } })
```

6. **\$rename – Renames a field**

Example:

```
db.users.updateOne({}, { $rename: { "fullName": "name" } })
```

7. **\$inc – Increases or decreases a numerical field**

Example:

```
db.users.updateOne({ name: "Alice" }, { $inc: { age: 1 } })
```

3.11 ELEVENTH WEEK

3.11.1: UPDATE ARRAY OPERATORS

In MongoDB, array update operators help modify arrays inside documents. Here are three important operators:

1. \$push – Adds an element to an array

Example:

```
db.users.updateOne({ name: "Alice" }, { $push: { skills: "MongoDB" } })
```

2. \$pop – Removes the first (-1) or last (1) element from an array

Example:

```
db.users.updateOne({ name: "Alice" }, { $pop: { skills: -1 } }) # Removes first
```

```
db.users.updateOne({ name: "Alice" }, { $pop: { skills: 1 } }) # Removes last  
item
```

3. \$pull – Removes a specific value from an array

Example:

```
db.users.updateOne({ name: "Alice" }, { $pull: { skills: "React" } })
```

3.12 TWELFTH WEEK

3.12.1 Node.js

Node JS is a free, open-source, cross-platform and Java script run-time environment that helps developers create servers, Webpage, command line tools, and scripts

Run time environment is where your program will be executed. All programming language is required compiler or interpreter to interrupt and execute your source code. All browser will have an inbuilt js engine that is capable of interpreting and executing your js code to make our website interactive.

Node provides one environment where you can execute your code outside of the browser.

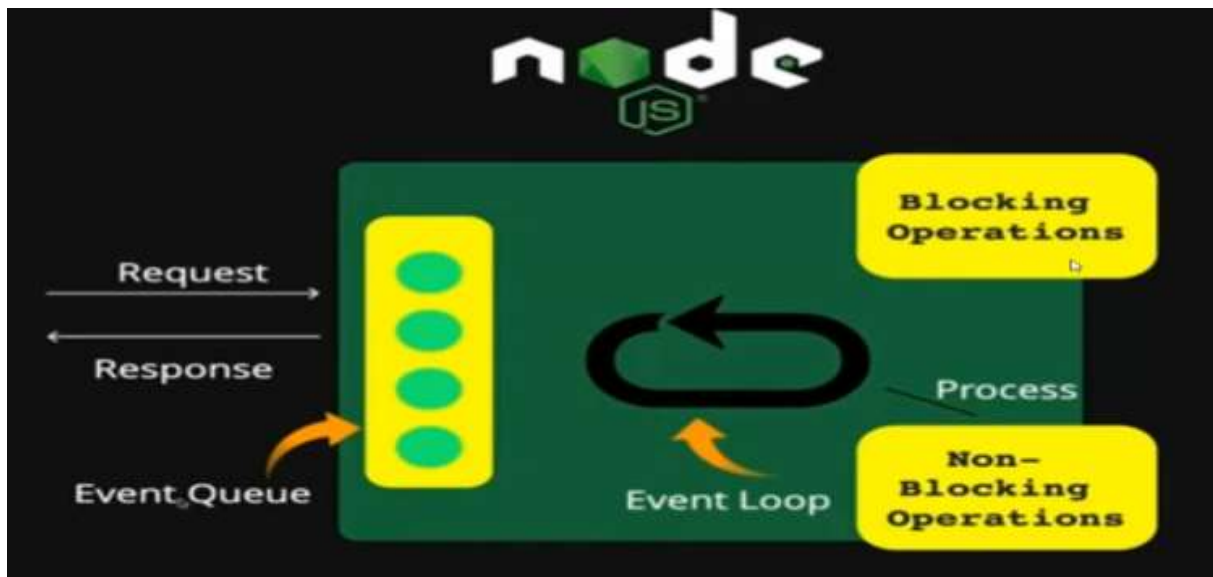


Fig 3.12: Nodejs

1. **Setting Up a Node.js Server:** The first step to backend development is setting up a Node.js server. Node.js allows you to handle HTTP requests and serve responses. This is often done using frameworks like Express.js.
2. **Routing in Express.js:** Routing is a mechanism to define how your server responds to different HTTP requests (GET, POST, PUT, DELETE). Routes are defined using the Express.js routing methods such as `app.get()`, `app.post()`, etc.
3. **Middleware in Express.js:** Middleware is a function that takes in the request object (`req`), the response object (`res`), and the next function. Middleware functions are used to perform operations like data validation, logging, or modifying requests before they reach the route handler.

3.12.2 REPL

REPL stands for Read Evaluate Print Loop and it is programming language environment. REPL session provides a convenient way write quickly execute javaScript code

R → Read It reads user given input.

E → Evaluate It takes user input and evaluate the data.

P → Print It prints the result.

L → Loop It loop the data until the session is expired

3.12.3 Node JS architecture

When the client sends request its stores it in event queue to execute sequentially.

It has two operations

1. **Blocking Operation (Synchronous):** Blocking operation executes the event one by one by blocking the event in hold. Blocking Operation uses thread pools to execute the event and return the result.
2. **Non-Blocking Operation (Asynchronous):** Non-Blocking operation executes all the events simultaneously.

3.12.4 Modules

Module is a simple functionality that can store in single file or multiple file.

In NodeJS there are three type of modules.

1. Local Module:

A local module is a module which is created in our system. Local module allow you to encapsulate related code within single file making it easy to manage. Function, Object, Class, Array defines in a local module can be reused in multiple files reducing the code of duplication.

2. Core Module:

NodeJS has many built-in modules that are part of the platform and come with NodeJS installation. These modules can be loaded into the program by using the require() function.

- **HTTP:** Creates HTTP server in NodeJS
- **Assert:** Set of assertion function useful for testing
- **FS (File System):** Used to handle file system.
- **Path:** Include method to deal with file path.

- **Process:** Provides information and control about NodeJS process
- **OS:** Provides info about operation system
- **Query String:** Utility used for parsing and formatting URL query string
- **URL:** Module provides utility for URL resolution and parsing.

3. Third-Party Module:

Third-party modules are external libraries or packages created by other developers or organizations that add functionality to your Node.js applications. These modules are not built into Node.js—you install them using npm (Node Package Manager).

3.12.5 File System

1. **copyFileSync() :**

syntax : fs.copyFileSync(src,dest)

this method is used to synchronously copy a file from source path to destination path

NodeJS will overwrite the file if it already exist in the destination

```
let res = fs.copyFileSync("./app.txt","./text.txt") console.log(res)
```

```
let res1 = fs.copyFileSync("./app.txt","./html.txt") console.log(res1)
```

2. **copyFile():**

syntax : fs.copyFile(src,dest,callback fuction)

this method is used to asynchronously copy a file from source path to destination path.

Node js will overwrite the file if it already exist in the destination.

```
let res = fs.copyFile("./html.txt","./js.txt",(error)⇒{
```

```
if(error) throw error
```

```
console.log("File created successfully")
```

```
}
```

```
console.log(res)
```

3. **existsSync():**

syntax : fs.existsSync("./app.txt")

fs.existsSync method in node is used to synchronously check if a file exists in a given path or not. it returns a Boolean value which indicates the existance of a file.

```
let res = fs.existsSync("./app.txt") console.log(res)
```

4. **exists():**

syntax : fs.exists("./filepath.txt",(exists)⇒{ clg(exists ? "file is present" : " file is not present")
})

fs.exists method in node is used to asynchronously check if a file exists in a given path or not. it returns a callback function which indicates the existence of a file.

```
let res = fs.exists("./app.txt",(exists)⇒{ console.log(exists ? "File is present" : " File is not present") }) console.log(res)
```

5. **renameSync():**

syntax : fs.renameSync("./filepath.txt","./filepath1.txt")

fs.renameSync method in node is used to synchronously change the name of the file

```
let res = fs.renameSync("./app.txt","app1.txt") console.log(res)
```

6. **rename():**

syntax : fs.rename("./filepath.txt","./filepath1.txt",()⇒{
if(err) throws err
console.log("File name changed")
})

fs.rename method in node is used to asynchronously change the name of the file

```
let res1 = fs.rename("./app.txt","app1.txt",()⇒{  
if(err) throws err  
console.log("File name Changed")  
})  
console.log(res1)
```

7. **writeFileSync():**

```
let res = fs.writeFileSync("./app1.txt"," hello everyone")  
console.log(res)
```

8. **writeFile():**

```
let res1 = fs.writeFile("./app1.txt"," hello everyone",(err)⇒{ if(err) throw err  
console.log("Message updated") }) console.log(res1)
```

3.13 THIRTEENTH WEEK

3.13.1 HTTP

It stands for Hyper Text Transfer Protocol The HTTP module in NodeJS provide functionality to create and manage HTTP servers and clients. It includes method to handle incoming requests, send responses and interact with HTTP headers and status code.

1. What are the HTTP status code

HTTP response status code indicate whether a specific HTTP request has been successfully completed.

```
let http = require("http")
```

```
console.log(http)
```

```
'100': 'Continue',
```

```
'101': 'Switching Protocols',
```

```
'102': 'Processing',
```

```
'103': 'Early Hints',
```

```
'200': 'OK',
```

```
'201': 'Created',
```

```
'202': 'Accepted',
```

```
'203': 'Non-Authoritative Information',
```

```
'204': 'No Content',
```

```
'205': 'Reset Content',
```

```
'206': 'Partial Content',
```

```
'207': 'Multi-Status',
```

```
'208': 'Already Reported',
```

```
'226': 'IM Used',
```

```
'300': 'Multiple Choices',
```

```
'301': 'Moved Permanently',
```

```
'302': 'Found',
```

```
'303': 'See Other',
```

```
'304': 'Not Modified',
```

```
'305': 'Use Proxy',
```

```
'307': 'Temporary Redirect',
```

'308': 'Permanent Redirect',
'400': 'Bad Request',
'401': 'Unauthorized',
'402': 'Payment Required',
'403': 'Forbidden',
'404': 'Not Found',
'405': 'Method Not Allowed',
'406': 'Not Acceptable',
'407': 'Proxy Authentication Required',
'408': 'Request Timeout',
'409': 'Conflict',
'410': 'Gone',
'411': 'Length Required',
'412': 'Precondition Failed',
'413': 'Payload Too Large',
'414': 'URI Too Long',
'415': 'Unsupported Media Type',
'416': 'Range Not Satisfiable',
'417': 'Expectation Failed',
'418': 'I'm a Teapot',
'421': 'Misdirected Request',
'422': 'Unprocessable Entity',
'423': 'Locked',
'424': 'Failed Dependency',
'425': 'Too Early',
'426': 'Upgrade Required',
'428': 'Precondition Required',
'429': 'Too Many Requests',
'431': 'Request Header Fields Too Large',
'451': 'Unavailable For Legal Reasons',
'500': 'Internal Server Error',
'501': 'Not Implemented',
'502': 'Bad Gateway',
'503': 'Service Unavailable',

'504': 'Gateway Timeout',
'505': 'HTTP Version Not Supported',
'506': 'Variant Also Negotiates',
'507': 'Insufficient Storage',
'508': 'Loop Detected',
'509': 'Bandwidth Limit Exceeded',
'510': 'Not Extended',
'511': 'Network Authentication Required'

2. Creating the server

```
let http = require("http")
let server = http.createServer((req,res)⇒{
    res.write("Hi i am NodeJS")
    res.end("OK")
})
server.listen(5000,(err)⇒{
    if(err) throw err
    console.log("server is running on :http://localhost:5000");
})
```

3. Conversion to JSON

```
let http = require("http")
let server = http.createServer((req,res)⇒{
    res.writeHead(200,{"content-type":"application/json"})
    let obj = {
        emp_name:"Sampad", emp_id:82
    }
    res.write(JSON.stringify(obj))
    res.end()
})
server.listen(4000,(err)⇒{
    if(err) throw err
    console.log("server is running on :http://localhost:4000")
})
```

4. http.createServer():

It creates an http server object. The http server object can listen to code on our computer and execute a function, a request listener each time a request is made.

5. server.listen():

The server.listen() creates a listener on specified port or path.

6. res.writeHead():

This property is used to send a response header to the incoming request. A status code represents a three-digit https request code and a header parameter contains the response header. Optionally a human-readable status message can be provided as a second argument.

7. res.write():

The res.write() is an in-built application programming interface of the http module which sends the chunk of response body that is omitted when the request is a HEAD REQUEST.

8. Status code:

'100': 'Continue',
'101': 'Switching Protocols',
'102': 'Processing',
'103': 'Early Hints',
'200': 'OK',
'201': 'Created',
'202': 'Accepted',
'203': 'Non-Authoritative Information',
'204': 'No Content',
'205': 'Reset Content',
'206': 'Partial Content',
'207': 'Multi-Status',
'208': 'Already Reported',
'226': 'IM Used',
'300': 'Multiple Choices',

'301': 'Moved Permanently',
'302': 'Found',
'303': 'See Other',
'304': 'Not Modified',
'305': 'Use Proxy',
'307': 'Temporary Redirect',
'308': 'Permanent Redirect',
'400': 'Bad Request',
'401': 'Unauthorized',
'402': 'Payment Required',
'403': 'Forbidden',
'404': 'Not Found',
'405': 'Method Not Allowed',
'406': 'Not Acceptable',
'407': 'Proxy Authentication Required',
'408': 'Request Timeout',
'409': 'Conflict',
'410': 'Gone',
'411': 'Length Required',
'412': 'Precondition Failed',
'413': 'Payload Too Large',
'414': 'URI Too Long',
'415': 'Unsupported Media Type',
'416': 'Range Not Satisfiable',
'417': 'Expectation Failed',
'418': 'I'm a Teapot',
'421': 'Misdirected Request',
'422': 'Unprocessable Entity',
'423': 'Locked',
'424': 'Failed Dependency',
'425': 'Too Early',
'426': 'Upgrade Required',
'428': 'Precondition Required',
'429': 'Too Many Requests',

'431': 'Request Header Fields Too Large',
'451': 'Unavailable For Legal Reasons',
'500': 'Internal Server Error',
'501': 'Not Implemented',
'502': 'Bad Gateway',
'503': 'Service Unavailable',
'504': 'Gateway Timeout',
'505': 'HTTP Version Not Supported',
'506': 'Variant Also Negotiates',
'507': 'Insufficient Storage',
'508': 'Loop Detected',
'509': 'Bandwidth Limit Exceeded',
'510': 'Not Extended',
'511': 'Network Authentication Required'

3.14 FOURTEENTH WEEK

3.14.1 Express JS

Express JS is a web application framework that provides you with a simple API to build websites, web apps and back ends. With ExpressJS, you need not worry about low level protocols, processes, etc.

3.14.2 Node Package Manager(npm)

npm is the package manager for node. The npm Registry is a public collection of packages of open-source code for Node.js, front-end web apps, mobile apps, robots, routers, and countless other needs of the JavaScript community. npm allows us to access all these packages and install them locally. You can browse through the list of packages available on npm

How to use npm?

There are two ways to install a package using npm: globally and locally

- **Globally:** This method is generally used to install development tools and CLI based packages. To install a package globally, use the following code.
`npm install -g <package-name>`

- **Locally:** This method is generally used to install frameworks and libraries. A locally installed package can be used only within the directory it is installed. To install a package locally, use the same command as above without the -g flag.
npm install <package-name>

3.14.3 EXPRESSJS – ROUTING

Web frameworks provide resources such as HTML pages, scripts, images, etc. at different routes.

The following function is used to define routes in an Express application:

- **app.method(path, handler)**

This METHOD can be applied to any one of the HTTP verbs – get, set, put, delete. An alternate method also exists, which executes independent of the request type. Path is the route at which the request will run.

3.14.4 EXPRESSJS – HTTP METHODS

Web frameworks provide resources such as HTML pages, scripts, images, etc. at different routes.

- **GET:** The GET method requests a representation of the specified resource. Requests using GET should only retrieve data and should have no other effect.
- **POST:** The POST method requests that the server accept the data enclosed in the request as a new object/entity of the resource identified by the URI.
- **PUT:** The PUT method requests that the server accept the data enclosed in the request as a modification to existing object identified by the URI. If it does not exist then the PUT method should create one.
- **DELETE:** The DELETE method requests that the server delete the specified resource.

3.14.5 EXPRESSJS – URL BUILDING

We can now define routes, but those are static or fixed. To use the dynamic routes, we SHOULD provide different types of routes. Using dynamic routes allows us to pass

parameters and process based on them.

Here is an example of a dynamic route:

```
var express = require('express');
var app = express();
app.get('/:id', function(req, res){
    res.send("The id you specified is " + req.params.id);
});
app.listen(3000);
```

3.15 FIFTEENTH WEEK

3.15.1 WORKED ON PROJECT

- During the final week of the internship, I worked on building a real-time chat application using the MERN stack in combination with Socket.IO for real-time, bidirectional communication between users. The project involved implementing features such as user authentication, online/offline user status indicators, private one-on-one messaging, and real-time message updates without page refreshes.
- The backend was built with Node.js and Express, using MongoDB for message and user data storage. React.js was used for the frontend, with Socket.IO integrated on both client and server sides to manage live communication.
- The project showcased the application of WebSocket technology to build interactive and responsive user experiences.
- This real-time chat application is a full-stack web app that allows users to register, log in, and exchange messages instantly without refreshing the page.
- It demonstrates practical implementation of **WebSockets** using **Socket.IO** in a **MERN stack** environment (MongoDB, Express.js, React.js, and Node.js).

CHAPTER 4

TASK COMPLETED

4.1 CALCULATOR USING HTML AND CSS

The task of designing a calculator using HTML and CSS focuses on creating a visually appealing and structured layout without incorporating functionality. The design includes essential elements such as a display screen to show inputs and outputs and buttons for digits (0–9), operators (+, -, *, /), and special keys like "C" (clear) and "=" (equals). HTML is used to build the structural framework of the calculator, while CSS is applied to enhance its appearance. Styling involves organizing the buttons and display screen, adding appropriate spacing, color schemes, fonts, hover effects, and ensuring a clean, responsive layout. The final result is a static mockup resembling a functional calculator, ideal for presentation or further development.

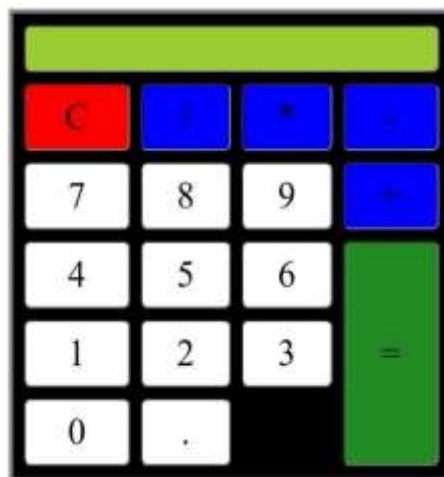


Fig 4.1: Calculator

4.2 TIME TABLE USING HTML AND CSS

The task of designing a timetable using HTML and CSS focuses on creating a structured and visually appealing layout to represent schedules or routines effectively. The design includes rows and columns to display days, time slots, and subjects or activities. HTML is used to build the framework of the table, with headers for days and time slots and cells to input relevant details. CSS is applied to enhance the appearance by adding styles such as borders, background colors, text alignment, padding, and hover effects. Additional styling ensures the timetable is clean, readable, and responsive, making it suitable for various devices. The final output is a static, well-organized timetable ideal for displaying schedules or plans.

Time Table					
Hours	Mon	Tues	Wed	Thurs	Fri
	Science	Maths	Science	Maths	Arts
	Science	Maths	Science	Maths	Arts
	Lunch				
	Science	Maths	Science	Maths	Project
	Science	Maths	Science	Maths	

Fig 4.2: Time Table

4.3 REGISTRATION FORM USING HTML AND CSS

The task of designing a registration form using HTML and CSS focuses on creating a user-friendly and visually appealing interface for collecting user information. The form includes essential fields such as name, email, phone number, password, and other inputs like dropdown menus, radio buttons, or checkboxes as needed. HTML is used to structure the form, including input fields, labels, and a submit button. CSS is applied to enhance the form's appearance by adding styles such as padding, margins, background colors, border designs, and responsive layouts. Additional styling features like hover effects, font selection, and error highlighting improve the overall user experience. The final result is a static, well-designed form ready for further development or integration.

Register Form

Please fill the form

Name

Email

Address

Password

PhoneNumber

Select Gender
 male ☐ female ☐

Select skills
 html ☐ CSS ☐ JavaScript ☐ MongoDB ☐ Nodejs ☐

Date:

Fig 4.3: Registration Form

4.4 BOUNCING BALL ANIMATION

The task of creating a bouncing ball animation using CSS focuses on designing a visually appealing and smooth motion effect to simulate a ball bouncing. HTML is used to structure the basic layout, defining a container and the ball element. CSS is then applied to style the ball, specifying its size, shape, and color. The animation is achieved using CSS keyframes, which define the ball's movement along the vertical axis to create the illusion of bouncing. Additional styles, such as easing effects, shadows, and gradients, enhance the visual appeal. The result is an engaging, dynamic animation showcasing the ball's motion without requiring any JavaScript, highlighting CSS's capabilities for creating interactive designs.

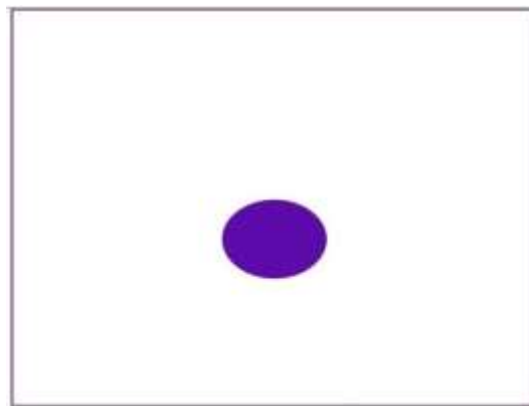


Fig 4.4: Bouncing Ball

4.5 HEARTBEAT ANIMATION

The task of creating a heartbeat animation using CSS focuses on designing a visually dynamic effect that simulates the rhythmic pulsing of a heart. HTML is used to structure the layout by defining a container and the heart element, typically created using shapes like a combination of circles and a square with CSS properties. CSS is then applied to style the heart, adding colors, gradients, and rounded corners for a realistic appearance. The animation is achieved using CSS keyframes, which define the heart's scaling effect to mimic the expansion and contraction of a heartbeat. Additional styling, such as smooth transitions and easing functions, enhances the visual effect.

The result is an engaging, responsive animation that replicates a heartbeat, showcasing CSS's ability to create captivating visual designs without requiring JavaScript.

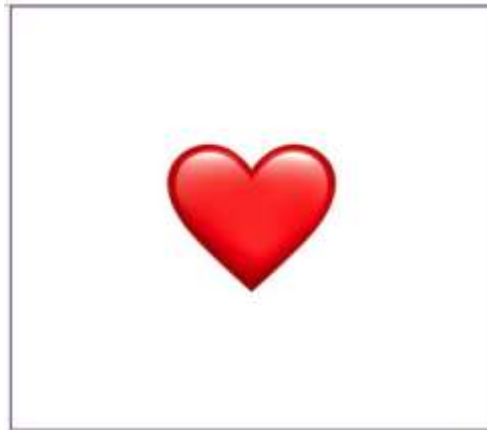


Fig 4.5: HeartBeat Animation

4.6 SQUARE BEAT ANIMATION

The task of creating a square beat animation using CSS focuses on designing a visually engaging effect where square elements pulsate rhythmically, simulating a "beating" motion. HTML is used to structure the layout by defining one or more square elements within a container. CSS is then applied to style the squares, specifying their size, color, and positioning. The animation is achieved using CSS keyframes, which define the scaling or opacity changes of the squares over time to create the beat effect. Easing functions and smooth transitions are added to enhance the animation's fluidity. The result is a dynamic and visually appealing animation that demonstrates CSS's capability to create rhythmic and interactive designs without additional scripting.

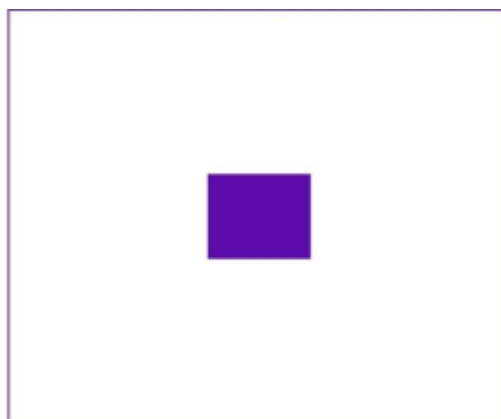


Fig 4.6: SquareBeat Animation

4.7 HALCYONIC WEBSITE CLONE

The task of creating a Halcyonic web clone using HTML and CSS involves replicating the design and layout of the Halcyonic website to produce a visually identical static version. HTML is used to structure the website's content, including elements like the header, navigation bar, sections, articles, and footer. CSS is applied to style these elements, ensuring they align with the original design in terms of colors, typography, spacing, and responsiveness. Additional CSS properties, such as flexbox or grid layouts, are used to achieve precise positioning and alignment of components. The focus is on accurately reproducing the visual aesthetics and responsive behavior of the original website without adding dynamic functionality. The result is a high-fidelity static clone showcasing attention to detail and proficiency in HTML and CSS.

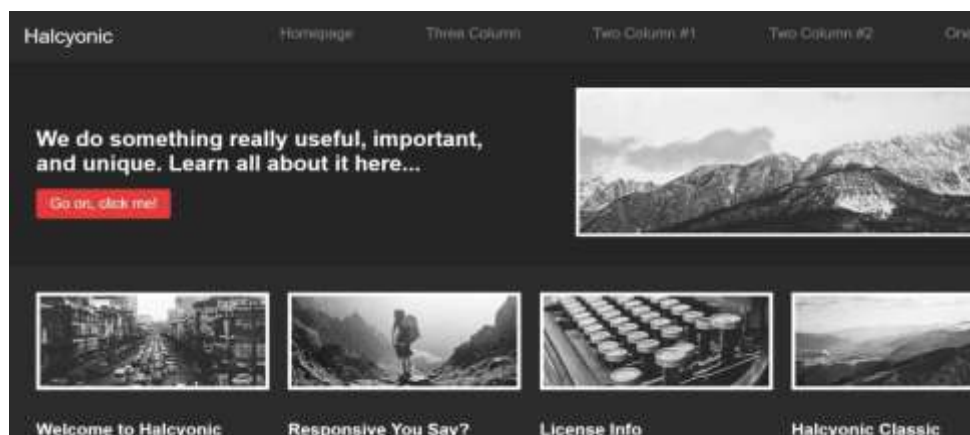


Fig 4.7: Website Clone

4.8 CHAMPIONS TROPHY

The task of creating the "Champions Trophy" team showcase involves designing an interactive and visually appealing layout to display player profiles using HTML, CSS, and JavaScript. Each player card includes an image, name, role (e.g., Batsman, Bowler, All-rounder), and a brief description of their contributions. HTML is used to structure the content, organizing player details into individual cards, while CSS enhances the appearance with consistent card styling, shadows, borders, typography, and a responsive grid layout. JavaScript adds interactivity, such as hover animations, dynamic filtering of players by roles (e.g., Batsman, Bowler), or pop-ups showing additional information when a card is clicked. The combined use of HTML, CSS, and JavaScript results in a professional, user-friendly design that is both functional and visually engaging, suitable for sports websites or event presentations.



Fig 4.8: Champion Trophy Playcard

4.9 DYNAMIC FOOD MENU DISPLAY WITH SEARCH FUNCTIONALITY

The below images showcases a food menu webpage where users can search for dishes, restaurants, or cuisine types using the search bar. The task involves dynamically fetching food details such as names, images, descriptions, and sources (e.g., restaurants) from a database or an API and displaying them as cards. Each card contains an image, name, description, and the source of the dish. The search functionality filters the displayed items in real time based on user input, improving user experience. The layout is designed using HTML for structure, CSS for styling, and JavaScript for functionality, ensuring responsiveness and ease of use. This feature can be integrated into restaurant or food delivery websites for enhanced customer interaction.

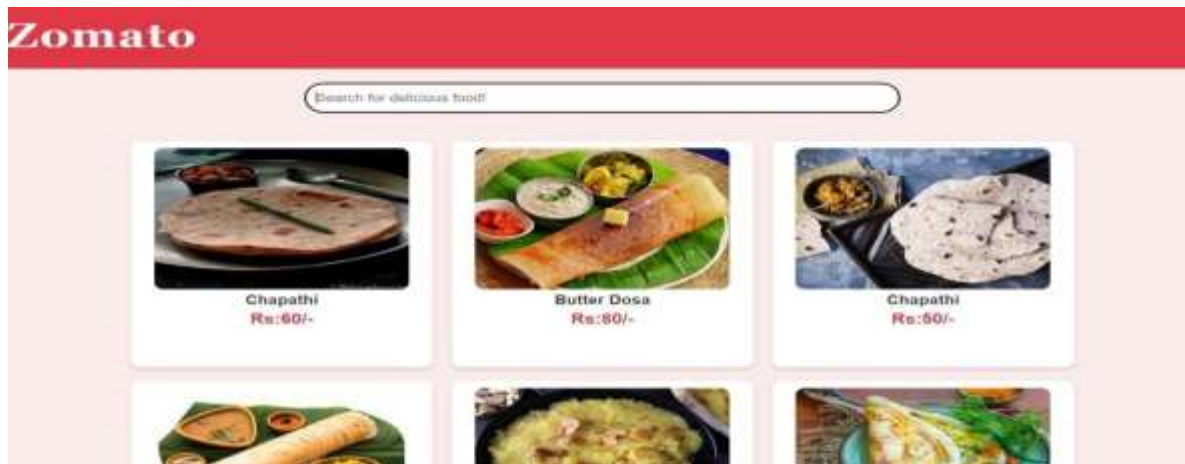


Fig 4.9: Dynamic Search Bar

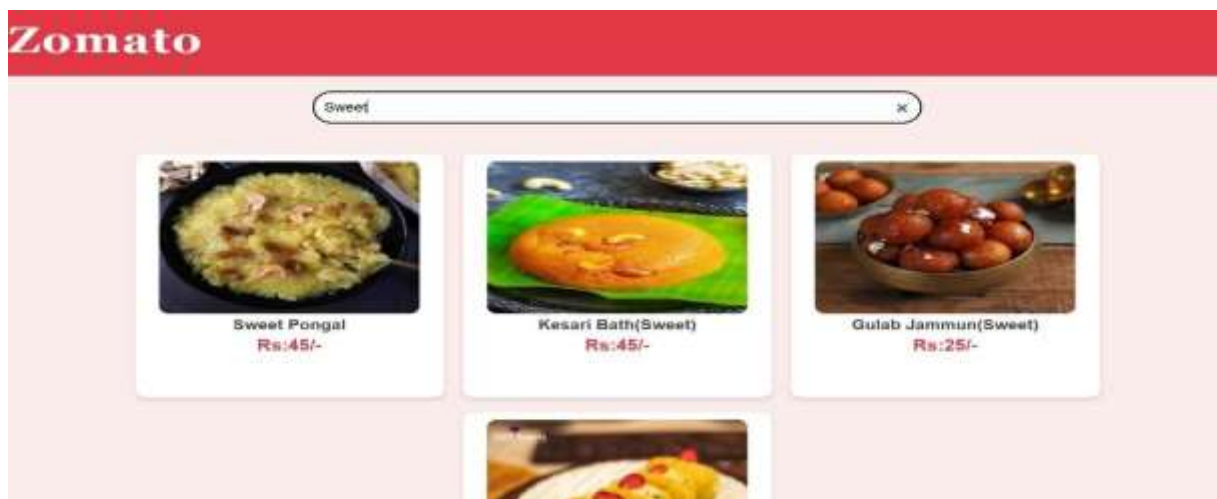


Fig 4.9.1: Result After Search

4.10 DYNAMIC PRODUCT SHOWCASE WITH FETCH API

The above image represents an online store webpage showcasing a collection of products dynamically displayed in a grid layout. The task involves fetching product details such as names, images, prices, descriptions, and categories from a database or an API and presenting them in a visually appealing and organized format. The webpage is built using HTML for structure, CSS for styling, and JavaScript for dynamic functionality. Each product card includes essential details like an image, title, price, description, and category. The layout is responsive, ensuring a user-friendly experience across devices. This approach allows real-time updates, enabling seamless integration of new products or modifications in the inventory.

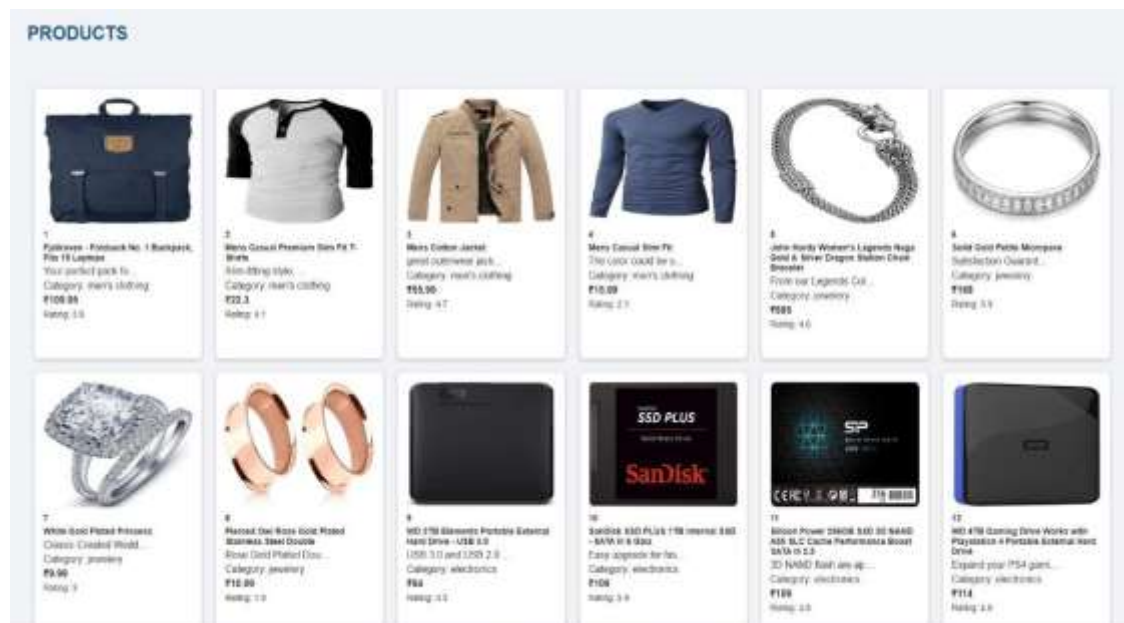


Fig 4.10 Dynamic Products Showcase

4.11 SONG DISPLAY USING REACT

Develop a simple React application to display a list of songs using a predefined dataset. The application should render song details such as title, artist, album (optional), and duration in a visually appealing tabular or card-based format. Use a responsive design to ensure compatibility across devices. Incorporate styling with CSS or a library like TailwindCSS to enhance the user interface. This task focuses on data display only, with optional features like headers or hover effects for improved user experience.

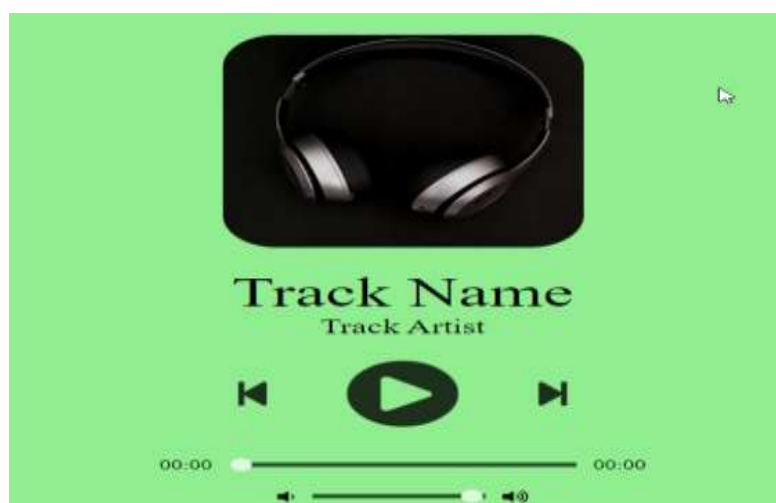


Fig 4.11 Song Display using React

4.12 LOGIN FORM USING REACT

Develop a simple and responsive login form using React. The form should include fields for username/email and password, along with a login button. Implement basic validation to ensure both fields are required, displaying error messages if left blank. Style the form using CSS or Tailwind CSS for a clean and modern look. Ensure the design is responsive for seamless use on desktop and mobile devices. Optionally, include features like a "Remember Me" checkbox and a "Forgot Password?" link for enhanced functionality.

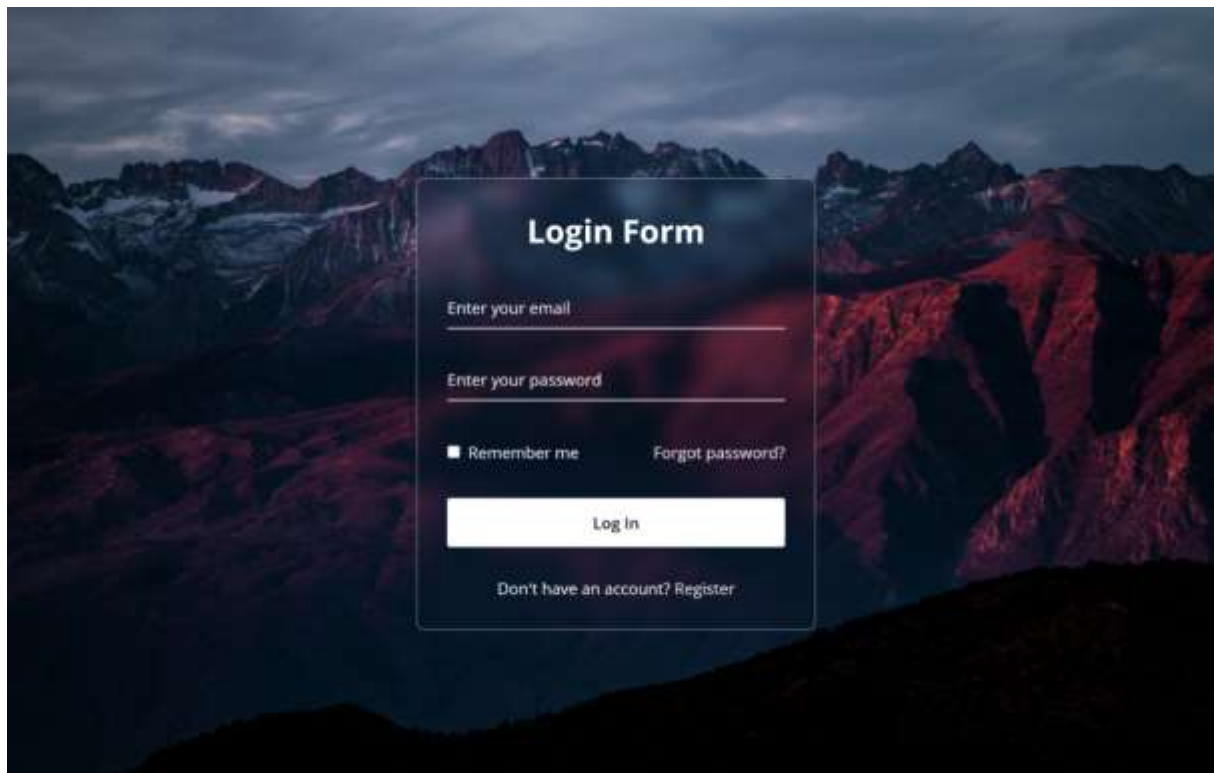


Fig 4.12:- Login Form using React

4.13 DESIGN A NAVIGATION BAR (NAVBAR) USING REACT

Create a responsive navigation bar using React that includes a brand logo or title on the left and navigation links such as "Home," "About," "Services," and "Contact." Use CSS or Tailwind CSS for a modern and visually appealing design, highlighting the active link for a better user experience. Ensure the navbar adapts to various screen sizes by implementing a collapsible or hamburger menu for smaller devices. Optionally, add interactive features such as hover effects, a search bar, or a user profile icon for enhanced functionality.

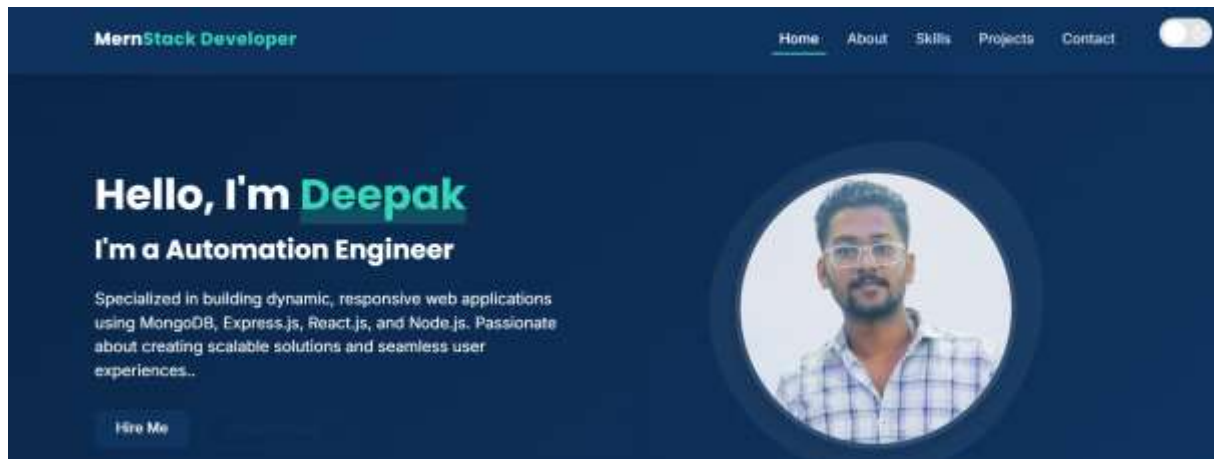


Fig 4.13:- Design a Navigation Bar (Navbar) using React

4.14 TO-DO LIST USING REACT

The **To-Do List** is a React-based task that enables users to efficiently add, delete, and mark tasks as completed. Each task is stored with a unique identifier and dynamically displayed within a structured list. Clicking on a task toggles its completion status, with a **strikethrough effect and green color** indicating that the task has been completed, ensuring clear visual feedback. Users can also remove tasks when necessary, providing an intuitive and responsive experience. Here by utilizing React hooks, particularly **use State**, the app maintains a dynamic, interactive experience where tasks update in real-time. **Event Handling** – Functions like handle Add Task, handle Delete Task, and toggle Task Status update the state based on user interactions. The outcome of this React-based To-Do List application is an efficient and user- friendly task management tool that allows users to organize their daily activities.

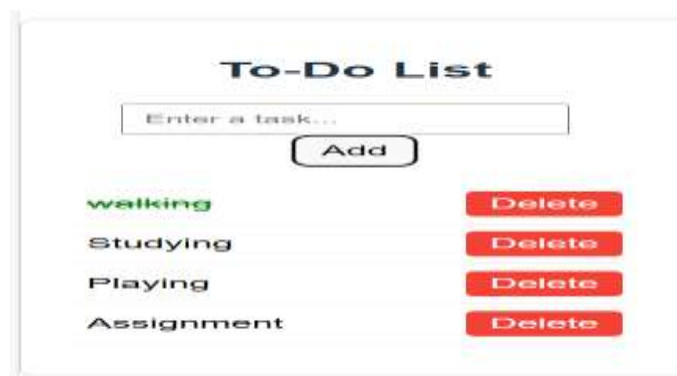


Fig 4.14:- To-Do List Using React Use State

CHAPTER 5

OUTCOMES

Real Time Chat Application

The goal of this project is to implement a full-stack real-time chat application that allows authenticated users to exchange messages instantly using WebSockets. This system demonstrates practical use of the MERN stack along with Socket.IO for bi-directional communication between clients.

5.1 Modules and Implementation

1. User Authentication

- **Registration:** Users register by providing a username, email, and password.
- **Login:** Users authenticate using credentials; JWT is issued and stored.
- **Security:** Passwords are hashed using **bcrypt** before saving in MongoDB.

2. Real-Time Messaging

- **WebSocket Setup:** Implemented using **Socket.IO** on both client and server sides.
- **User Connection:** Socket establishes a connection when the user logs in.
- **Message Sending:**
 - Sender emits a `sendMessage` event.
 - Server receives it and relays it to the recipient in real-time using their socket ID.
 - Message is also stored in MongoDB.

3. Chat Interface (Frontend - React)

- **Chat Room:** Displays user list and allows selecting a user to chat with.
- **Message Window:** Displays chat history using messages fetched via Axios.
- **Live Typing:** Socket emits and listens for typing indicators (optional).
- **Status Indicator:** Shows online/offline status in real-time.

4. Backend API (Express.js)

- **User Routes:**
 - `POST /register`: Registers a new user.

- POST /login: Authenticates user and returns JWT.
- GET /users: Returns list of all users excluding self.
- **Message Routes:**
 - POST /message: Sends and stores a message.
 - GET /messages/:userId: Retrieves conversation history with a specific user.

5. MongoDB Integration

- **Users Collection:**
 - Stores name, email, hashed password, and socket ID.
- **Messages Collection:**
 - Stores sender ID, receiver ID, message text, and timestamp.

6. Socket.IO Event Handling

Connection	→	Registers new client connection
Join	→	Adds user to online users map
SendMessage	→	Transfers message to the recipient
Disconnect	→	Removes user from online map

5.2 Results

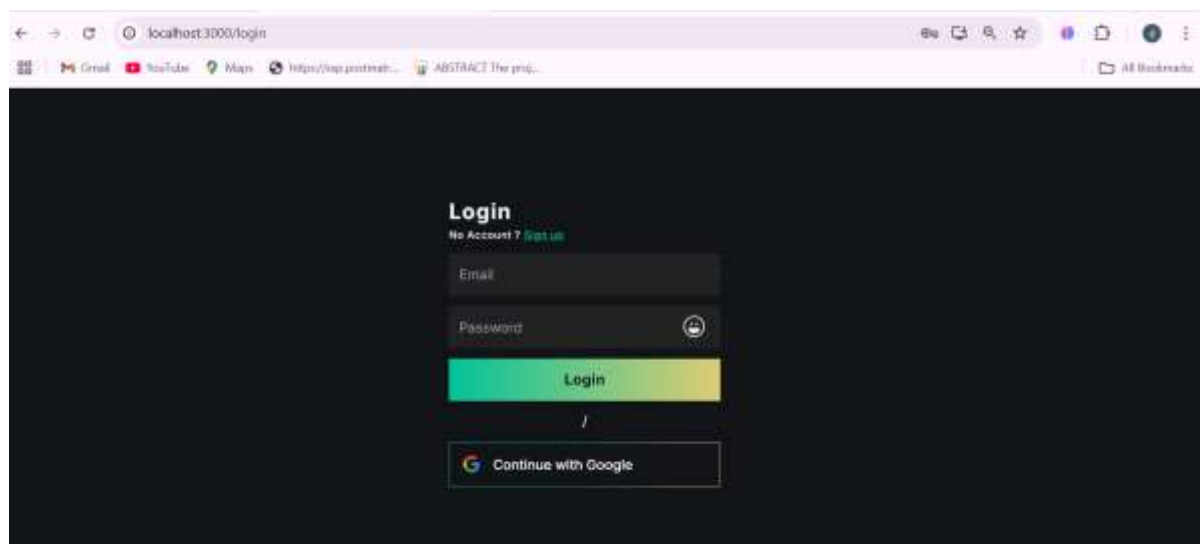


Fig 5.1 User Login Page

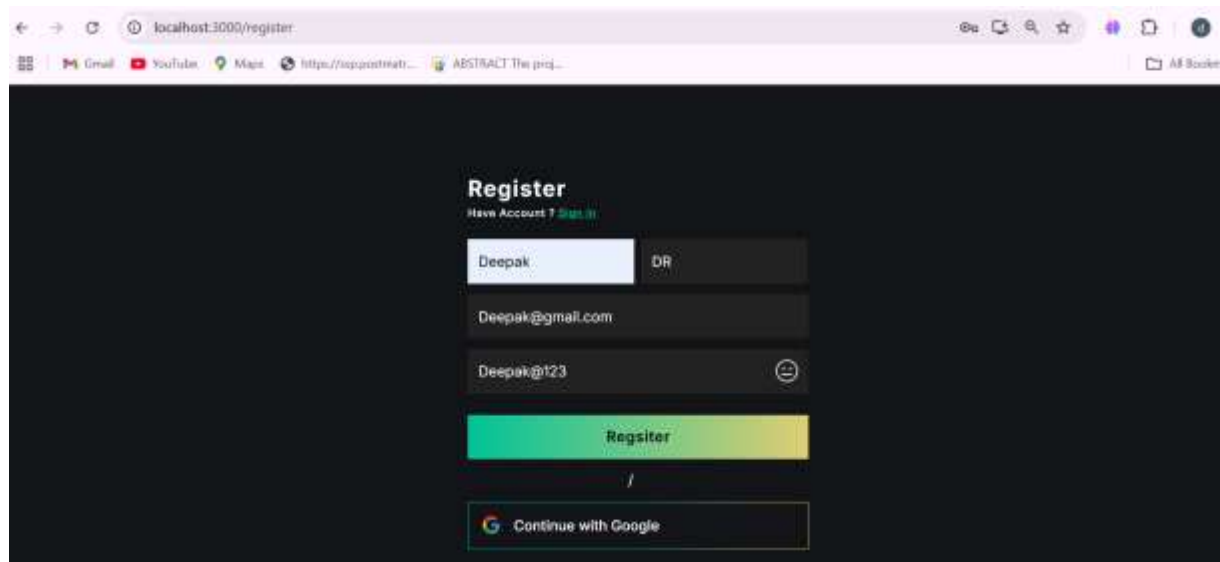


Fig 5.2 User Registration page

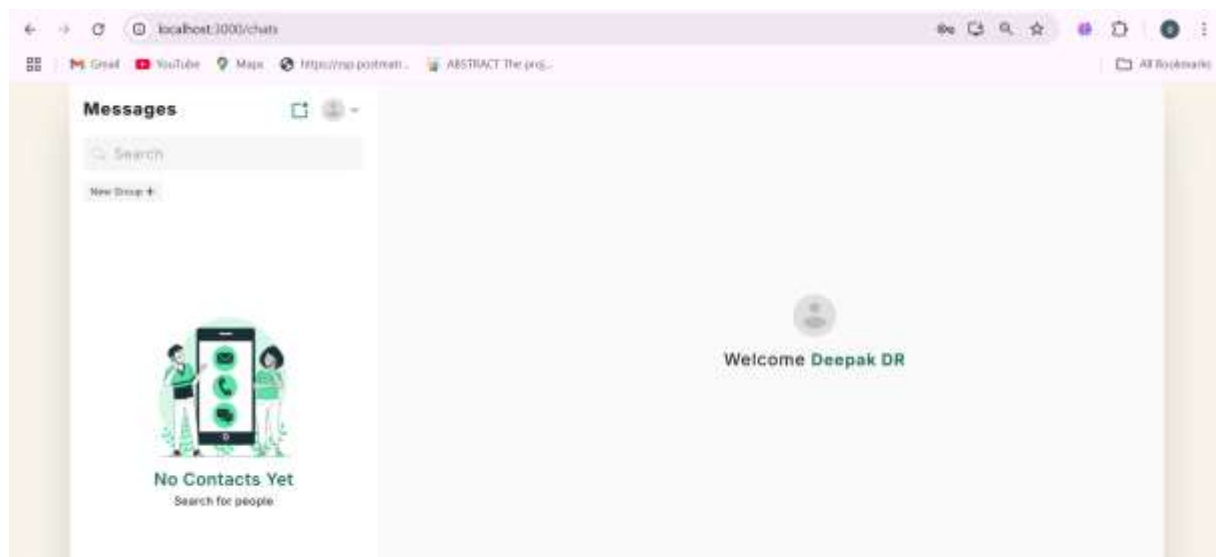


Fig 5.3 User Chat Application Dashboard

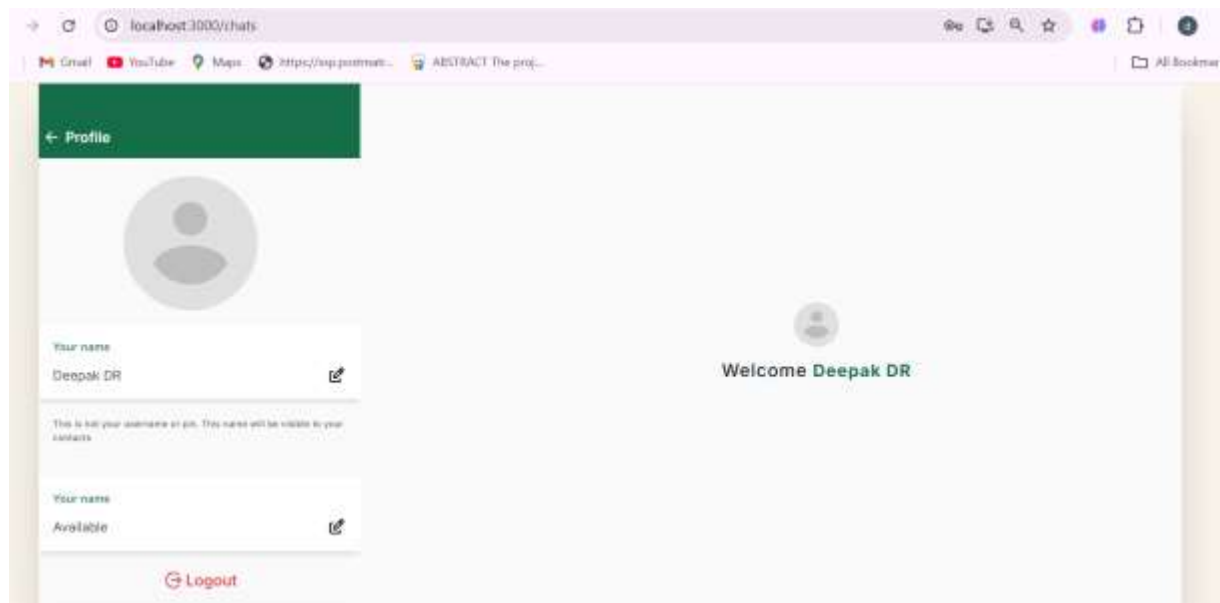


Fig 5.4 User Profile

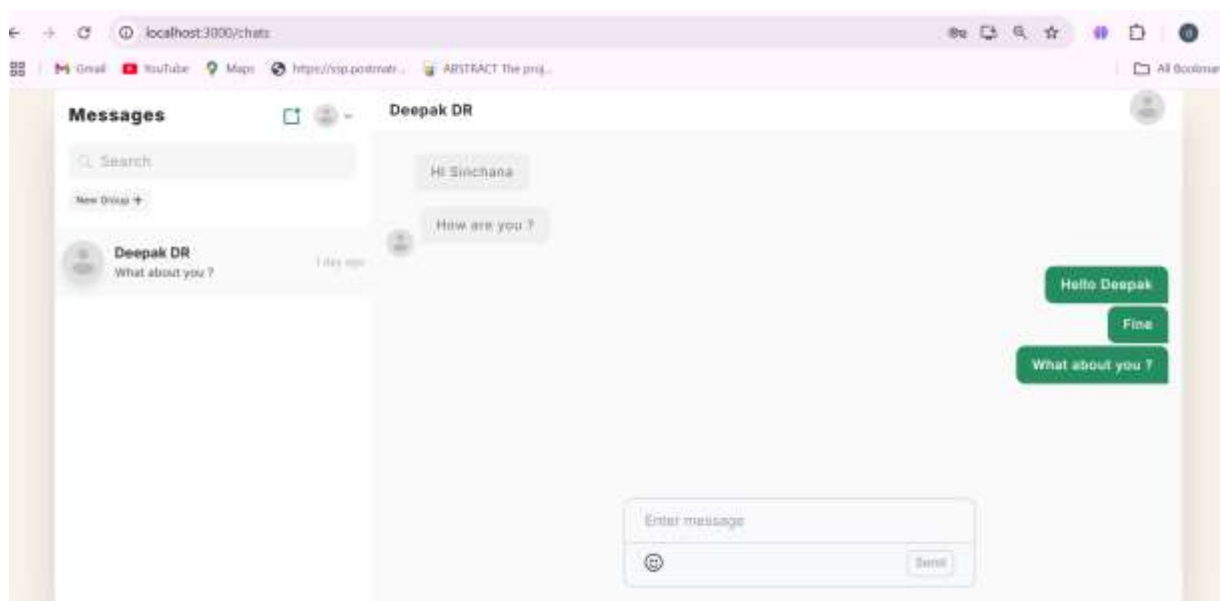


Fig 5.5 Chat from Deepak's Perspective

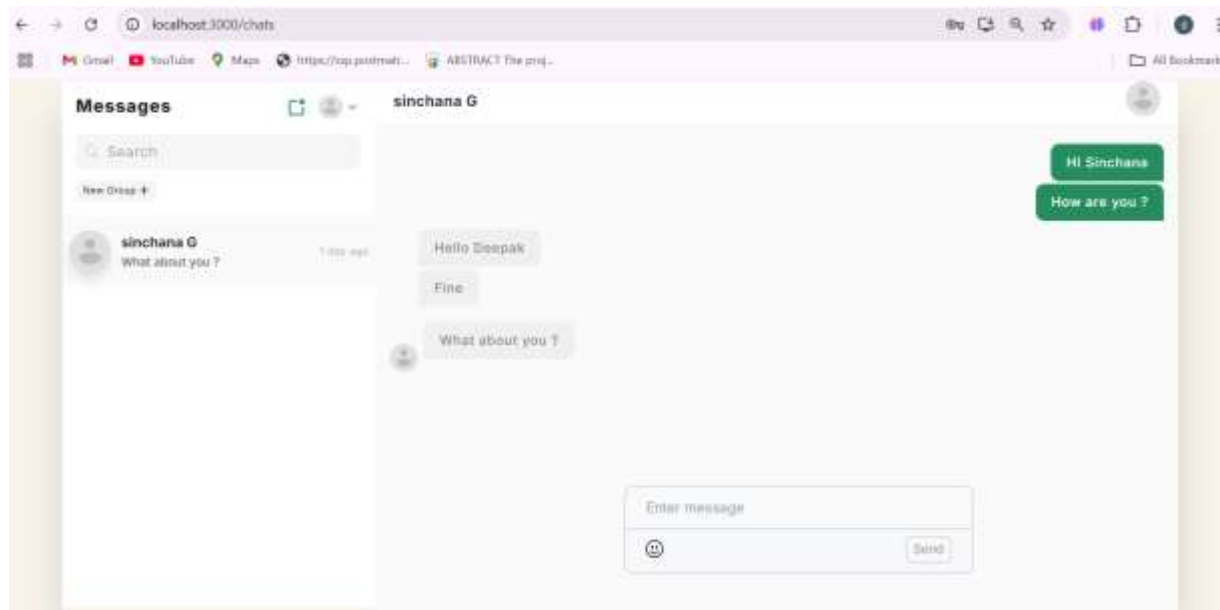


Fig 5.6 Chat from Sinchana's Perspective

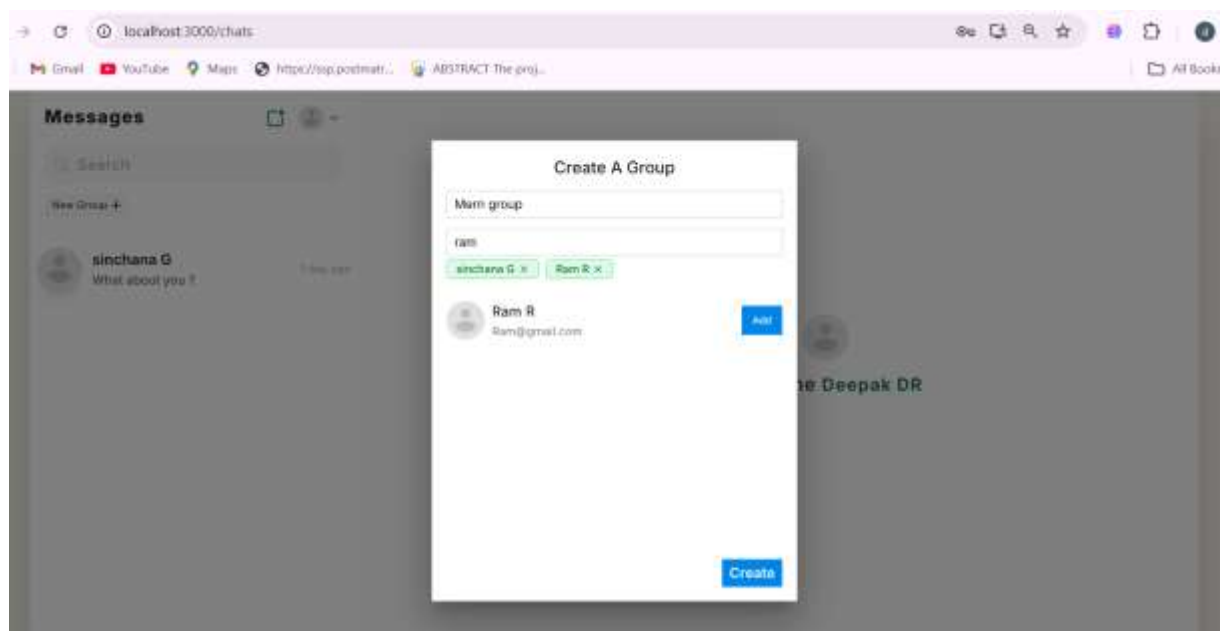


Fig 5.7 Group Creation Interface in Chat Application

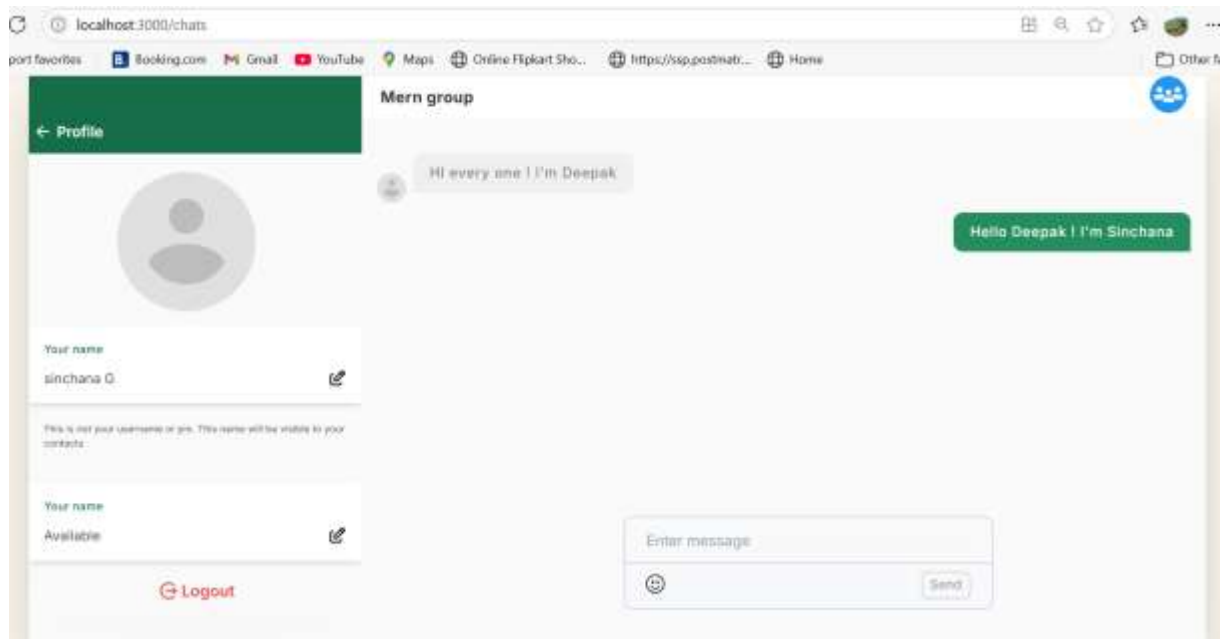


Fig 5.8 Group chat from Sinchana's Perspective

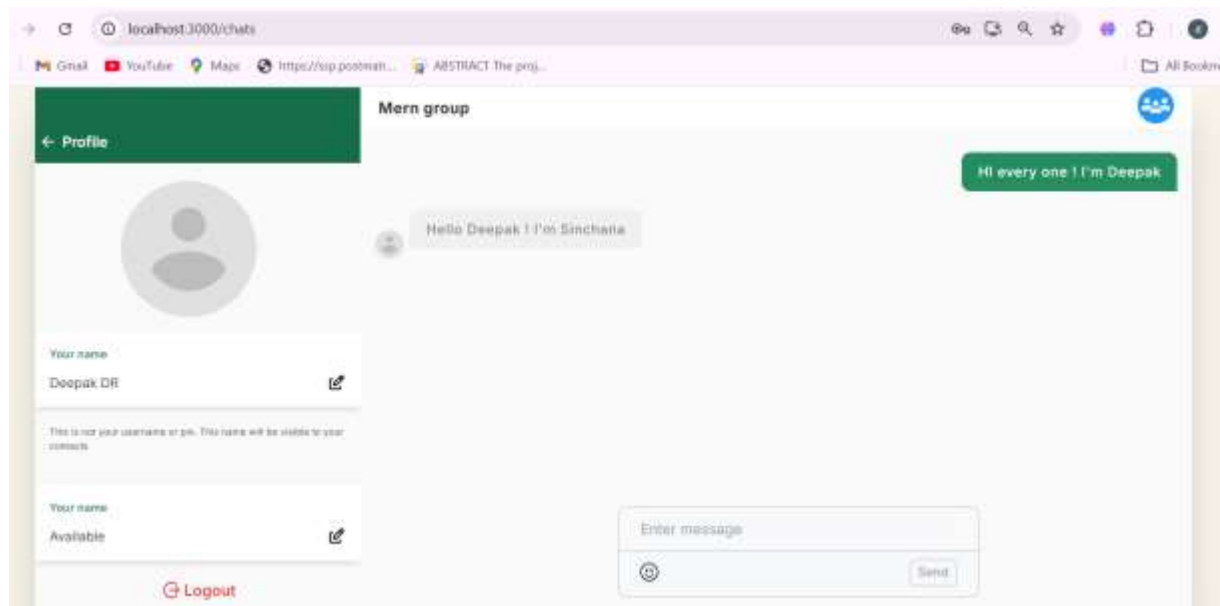


Fig 5.9 Deepak's View - Initiating the Conversation in MERN Group Chat

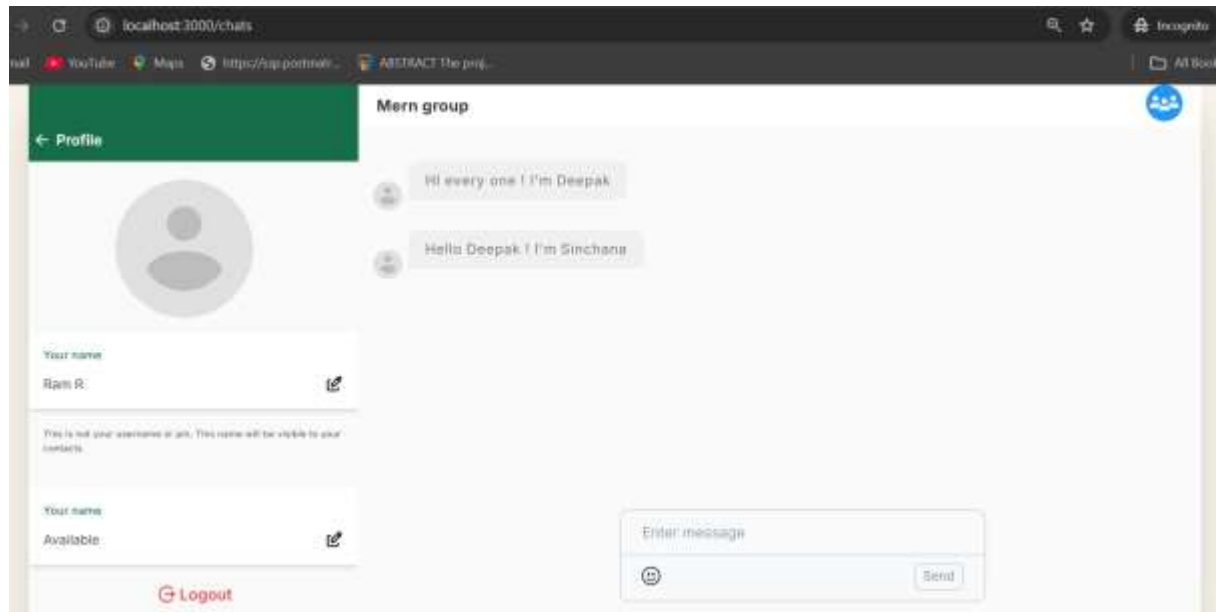


Fig 5.10 Ram Observes – A Third-User View of Synchronized Group Messaging

CHAPTER 6

CONCLUSION

The Realtime Chat application is a full-featured real-time messaging platform built using the MERN stack and Socket.IO. It includes essential chat functionalities such as user authentication, one-to-one and group chat, message notifications, real-time message exchange, emoji reactions, and user presence status. These features provide a smooth, engaging, and responsive user experience.

This project highlights proficiency in building scalable and interactive web applications and effectively combines frontend and backend technologies with real-time communication mechanisms.

Future Enhancements:

1. **Typing Indicator**

Show a “user is typing...” message to enhance real-time interactivity.

2. **Media/File Sharing**

Allow users to send images, videos, or documents through the chat.

3. **Read Receipts**

Add indicators for when messages are delivered and seen by the recipient.

4. **Dark Mode**

Provide a dark/light mode toggle to improve accessibility and user preference.

5. **Voice & Video Calling**

Integrate WebRTC for real-time voice or video communication within the chat

REFERENCE

- [1]. **React Documentation** – Retrieved from <https://react.dev/learn>
- [2]. **Node.js Documentation** – Retrieved from <https://nodejs.org/en/docs>
- [3]. **Express.js Documentation** – Retrieved from <https://expressjs.com/en/4x/api.html>
- [4]. **MongoDB Documentation** – Retrieved from <https://docs.mongodb.com/manual>
- [5]. **Socket.io Documentation**- Retrieved from <https://socket.io/docs/v4>
- [6]. **W3Schools**- Retrieved from <https://www.w3schools.com>
- [7]. **Emoji Mart**- Retrieved from <https://www.npmjs.com/package/emoji-mart>